从 3DGS CUDA 优化文章到 gsplat

逐点讲解、数学推导、代码示例与 RTX 4090 实测

基于 gsplat CUDA 源码与完整场景实验

2026 年 10 月

读者导言

这份材料不是四页摘要，也不是 CUDA 优化文章的逐字翻译。它按文章的背景、流水线、文献基线、Step 2–6、失败实验和硬件分析逐点展开，并把每个思想映射到本次 gsplat 源码和 RTX 4090 结果。每一项都注明是“仓库已有”“本次保留”“实验后回滚”还是“只分析、未移植”。

最重要的边界先说清楚：最终只保留三项 backward raster 改动；本场景为 20,789,814 个 Gaussian、degree-1 SH；GPU 是 RTX 4090，不是文章的 A100。RGB raster-only 三视角加速为 1.337x、1.648x、1.363x；RGB+ED/absgrad 在 camera 0 为 1.311x raster、1.081x full render。图像/alpha 完全相同，但单次 raw quaternion 梯度相对 L2 约 1.63e-4，超过 1e-4 严格阈值。固定 16 次平均梯度通过并不能抹掉这个单次失败，也不证明训练收敛。

源码/实验交付：gsplat commit 88513ea；gsplat implementation commit 88513ea。完整工程日志为 `benchmarks/blog_eval/REPORT.md`，本教材则重点解释“为什么”。

Contents

1 数学对象与 CUDA 执行模型 **1**

1.1 先建立数学对象：一个 Gaussian 究竟存了什么? 1

1.1.1 位置、形状、不透明度与颜色 1

1.1.2 为什么训练参数使用 log-scale 和 logit? 1

1.1.3 四元数的顺序、归一化和梯度 2

1.2 从三维椭球推导屏幕椭圆 2

1.2.1 相机变换与透视 Jacobian 2

1.2.2 求逆以后为什么叫 conic? 3

1.2.3 阈值如何变成 tile culling 条件? 3

1.3 Alpha compositing：颜色权重来自哪里? 3

1.3.1 前向递推与背景 3

1.3.2 阈值是渲染语义，不只是性能开关 4

1.3.3 手算一个有背景的例子 4

1.4 Reverse compositing 与 VJP：为什么反向要倒着走? 4

1.4.1 VJP 不是构造一个巨大 Jacobian 4

1.4.2 恢复透射率与尾部裁剪 5

1.4.3 从 alpha 梯度回到 conic 和中心 5

1.4.4 用手算例子检验反向直觉 6

1.5 可运行的最小 Python 数学实验 6

1.6 把数学放到 GPU：线程、存储和同步 7

1.6.1 Block、warp 和 lane 的层级 7

1.6.2 寄存器、shared memory、L2 和显存 7

1.6.3 为什么 backward 需要 atomic add? 8

1.7 CUDA 教学伪代码：协作加载与像素递推 8

1.8 SIMT 分歧：少一个分支为什么不一定更快? 10

1.9 如何读懂 RTX 4090 上的计时：roofline 与 Amdahl 10

1.9.1 Roofline 是上界模型，不是自动诊断 10

1.9.2 Amdahl：raster 加速不能直接当成完整渲染加速 10

1.9.3 异步执行与可比较的实验条件 11

1.10 把本章公式映射回优化问题 11

2 原文章逐点解读：S00–S06 **12**

2.1 阅读地图：把文章当作一条可验证的优化链 . . . 12
2.1.1 先建立一个关于成本的模型 . . . 12
2.2 s00：忠实基线比一个慢基线更重要 . . . 13
2.2.1 五阶段流水线和不可随意改变的语义 . . . 13
2.2.2 反向为什么昂贵 . . . 13
2.3 s01：先把五类已有文献优化讲透 . . . 14
2.3.1 第一类：warp butterfly 归约，先少写再谈快算 . . . 14
2.3.2 第二类：不透明度感知的精确行跨度 . . . 14
2.3.3 第三类：8×4 子块掩码和 48 字节结构体 . . . 15
2.3.4 第四类：两阶段排序，把重复的深度工作提到前面 . . . 16
2.3.5 第五类：少做 SH、宽加载、base-2 指数与标量反向状态 . . . 16
2.4 s02：让归约形状匹配梯度宽度 . . . 18
2.4.1 halving reduce-scatter 的核心不变量 . . . 18
2.4.2 两 splat half-warp folding：填满尾部的空闲位置 . . . 18
2.5 s03：SH 寄存器流式化与协作清零 . . . 19
2.5.1 为什么 SH 不需要同时保存全部输入和输出 . . . 19
2.5.2 移除 bulk memset 不是允许输出未初始化 . . . 19
2.6 s04：前向知道谁贡献过，就不要让反向重新猜 . . . 20
2.6.1 几何重叠与真实贡献的区别 . . . 20
2.6.2 原位压缩和停止索引的重映射 . . . 20
2.6.3 高字节 cmask 与全局 touched . . . 21
2.6.4 本任务选择更小的改动：只裁掉完全无用的尾部 . . . 21
2.7 s05：分歧、谓词化与精确的数值分支边界 . . . 21
2.7.1 为什么 “少执行线程” 未必意味着少发射指令 . . . 21
2.7.2 精确舍入：前后向必须在同一侧跨过 cutoff . . . 22
2.7.3 clamp、cutoff 和反向零贡献不是同一件事 . . . 22
2.8 s06：多行映射与执行流水线平衡 . . . 23
2.8.1 每线程两个竖直像素：先在线程内合并 . . . 23
2.8.2 warp 私有 staging：去掉等待最慢邻居的时间 . . . 23
2.8.3 把不透明度并入指数，复用列常数 . . . 24
2.8.4 竖直两行矩：把乘法延后到归约之后 . . . 24
2.8.5 终止状态编码与 BF16 的不同性质 . . . 24
2.9 负结果：失败实验教会了什么 . . . 24
2.9.1 驻留率不是越高越好，寄存器也不是越少越好 . . . 25
2.9.2 共享内存缓存并不免费，更多并行也可能引入等待 . . . 25
2.9.3 缓存局部性和排序的收益会互相影响 . . . 25
2.9.4 本任务三个明确撤回的实验 . . . 25
2.10 把文章读完之后：怎样评价这次移植 . . . 26
2.10.1 交付范围与最终测量 . . . 26
2.10.2 正确性证据也需要分层 . . . 26
2.10.3 一套可以带走的思考顺序 . . . 26

3 从纸面公式到可运行的最小例子 27
3.1 动手算例：从一个像素到一次完整渲染 . . . 27
3.1.1 算例一：从三维输入到 conic，再算两个 splat 的合成 . . . 27
3.1.2 算例二：四 lane、四分量的归约与 reduce-scatter . . . 28
3.1.3 算例三：把 ownership 映射到完整 CUDA warp . . . 29
3.1.4 算例四：Amdahl 模型解释完整渲染为什么只快一点 . . . 30
3.1.5 算例五：对齐 benchmark，并计算 relL2 . . . 31
3.1.6 算例六：安全 predication 不是最后乘一个零 . . . 32

4 gsplat 移植、评估和最终结论 34
4.1 从文章到 gsplat：保留的是哪些实现 . . . 34
4.1.1 先找到源码中的执行边界 . . . 34
4.1.2 Reduce-scatter：把结果所有权分散到 lane . . . 34
4.1.3 4×4 tile 暴露的 partial-warp 错误 . . . 35
4.1.4 后缀裁剪：从像素停止位置推导安全加载终点 . . . 35
4.1.5 安全 predication：无效 lane 的零必须是真的零 . . . 36
4.2 拒绝实验也是结果，而不是漏掉的步骤 . . . 36
4.2.1 三个已测试、未保留的候选 . . . 36
4.2.2 没有实现，就没有可引用的收益 . . . 37
4.3 机器评估：先证明比较的是同一个问题 . . . 37
4.3.1 输入、二进制与计时协议 . . . 37
4.3.2 重建后的最终时间：RGB 三个相机 . . . 38
4.3.3 产品配置：RGB+ED、absgrad、相机 0 . . . 38
4.4 正确性必须分层作答 . . . 39
4.4.1 单次梯度：屏幕空间通过，原始四元数仍失败 . . . 39
4.4.2 预先固定 16 次均值：回答的是另一个问题 . . . 39
4.4.3 136 个差分案例与两个 sanitizer . . . 39

Chapter 1

数学对象与 CUDA 执行模型

1.1 先建立数学对象：一个 Gaussian 究竟存了什么?

本章为阅读 3DGS CUDA 优化文章和本仓库的 gsplat 对照实验补齐基础。目标不是把公式背下来，而是能够回答三个问题：某个线程正在计算哪一个数学量，它从哪里读数据，以及改变执行顺序以后为什么图像可能不变、梯度却略有不同。下文的数学模型首先描述理想运算，再单独指出离散排序、阈值和浮点数带来的实现边界。所有教学代码均为解释用的独立例子，不是从 gsplat 源码逐行摘录的实现。

1.1.1 位置、形状、不透明度与颜色

第 i 个三维 Gaussian 的中心为 $\boldsymbol{\mu}_i \in \mathbb{R}^3$，形状由对称正定矩阵 $\Sigma_i \in \mathbb{R}^{3\times 3}$ 表示。相对于中心的位移 $\mathbf{x} - \boldsymbol{\mu}_i$ 越大，其椭球度量距离通常越大：

$$G_i(\mathbf{x}) = \exp\left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^T\Sigma_i^{-1}(\mathbf{x}-\boldsymbol{\mu}_i)\right]. \tag{1.1}$$

这里没有概率密度的归一化因子 $(2\pi)^{-3/2}|\Sigma_i|^{-1/2}$。3DGS 中的 Gaussian 是可学习的渲染基元，不应直接理解成总概率质量为一的分布。它的峰值强度由另一个不透明度参数 o_i 控制。屏幕 Gaussian 同样通常采用未归一化的指数核，而非对三维概率密度做精确射线积分。

颜色可以直接存成 RGB 向量 $\mathbf{c}_i$，也可以存成球谐函数系数：

$$\mathbf{c}_i(\mathbf{v}) = \sum_{\ell=0}^{L}\sum_{m=-\ell}^{\ell}\boldsymbol{\beta}_{i,\ell m}Y_{\ell m}(\mathbf{v}). \tag{1.2}$$

其中 $\mathbf{v}$ 为观察方向，$\boldsymbol{\beta}_{i,\ell m} \in \mathbb{R}^3$。每个颜色通道有 $(L+1)^2$ 个系数；degree-1 是 $4\times 3 = 12$ 个标量，degree-3 是 $16\times 3 = 48$ 个标量。颜色偏置、截断以及观察方向的正负约定要查具体 API。这一区别直接影响寄存器占用和读取量，不能把高阶 SH 场景中的优化幅度照搬到本次 degree-1 场景。

1.1.2 为什么训练参数使用 log-scale 和 logit?

用三个正尺度 s_x, s_y, s_z 和旋转矩阵 R 表示协方差：

$$\Sigma = R\,\mathrm{diag}(s_x^2, s_y^2, s_z^2)R^T. \tag{1.3}$$

训练时常存储无约束的 $\ell_j \in \mathbb{R}$，再令

$$s_j = \exp(\ell_j), \qquad o = \mathrm{sigmoid}(\eta) = \frac{1}{1+\exp(-\eta)}. \tag{1.4}$$

这样优化器不会产生负尺度，且有限的 η 对应 $0 < o < 1$。注意 ℓ_j 是尺度的对数，不是方差的对数，所以协方差中的对应对角项为 $\exp(2\ell_j)$，不能漏掉这个二倍因子。

链式法则给出

$$\frac{\partial L}{\partial \ell_j} = s_j \frac{\partial L}{\partial s_j}, \qquad \frac{\partial L}{\partial \eta} = o(1-o)\frac{\partial L}{\partial o}. \tag{1.5}$$

因此屏幕空间 opacity 梯度、激活后的 opacity 梯度和原始 logit 梯度不是同一个量。gsplat 渲染接口中的 scales 和 opacities 通常是调用方已经激活的值；不能因为训练器保存 log 参数，就在底层 kernel 中再做一次指数或 sigmoid。

1.1.3 四元数的顺序、归一化和梯度

本章采用 gsplat 文档中的 $\mathbf{q} = (w, x, y, z)$ 顺序。旋转矩阵使用单位四元数 $\widehat{\mathbf{q}} = \mathbf{q}/\|\mathbf{q}\|$。将归一化后的分量仍记为 w, x, y, z，则

$$R = \begin{bmatrix} 1-2(y^2+z^2) & 2(xy-wz) & 2(xz+wy) \\ 2(xy+wz) & 1-2(x^2+z^2) & 2(yz-wx) \\ 2(xz-wy) & 2(yz+wx) & 1-2(x^2+y^2) \end{bmatrix}. \tag{1.6}$$

这规定了本文的旋转约定；接入其他库时还要核对主动旋转、坐标轴方向及矩阵乘法顺序。$\mathbf{q}$ 与 $-\mathbf{q}$ 表示同一旋转，但原始参数并不相同。零四元数不能直接归一化，需要初始化或显式的数值保护。

对非零原始参数，归一化的 Jacobian 为

$$\frac{\partial \widehat{\mathbf{q}}}{\partial \mathbf{q}} = \frac{1}{\|\mathbf{q}\|}\left(I - \widehat{\mathbf{q}}\widehat{\mathbf{q}}^T\right). \tag{1.7}$$

所以 raw quaternion 梯度还经过了旋转、协方差、投影和归一化等多层链式法则。接近零范数的参数会带来额外敏感性。即使 raster backward 的局部梯度很接近，也不能据此宣称原始四元数梯度必然满足同一相对误差门槛。

1.2 从三维椭球推导屏幕椭圆

1.2.1 相机变换与透视 Jacobian

设世界到相机的变换为

$$\mathbf{x}_c = W\mathbf{x}_w + \mathbf{t}, \qquad \boldsymbol{\mu}_c = W\boldsymbol{\mu} + \mathbf{t} = (X, Y, Z)^T. \tag{1.8}$$

这里 W 只是 3×3 的旋转部分，不是完整的 4×4 齐次矩阵。平移改变中心，但不会改变协方差，因此 $\Sigma_c = W\Sigma W^T$。对于无畸变针孔相机、相机前方 $Z > 0$，屏幕投影为

$$\pi(X, Y, Z) = \begin{bmatrix} f_x X/Z + c_x \\ f_y Y/Z + c_y \end{bmatrix}, \qquad J = \left.\frac{\partial \pi}{\partial \mathbf{x}_c}\right|_{\boldsymbol{\mu}_c} = \begin{bmatrix} f_x/Z & 0 & -f_x X/Z^2 \\ 0 & f_y/Z & -f_y Y/Z^2 \end{bmatrix}. \tag{1.9}$$

例如 $\partial(f_x X/Z)/\partial Z = -f_x X/Z^2$：深度变化不仅改变物体大小，也影响偏离光轴的点的屏幕位置。仅保留 $f_x/Z, f_y/Z$ 两项会漏掉这个耦合。

在中心附近写 $\mathbf{x}_c = \boldsymbol{\mu}_c + \delta\mathbf{x}$，一阶近似给出

$$\pi(\boldsymbol{\mu}_c + \delta\mathbf{x}) \approx \pi(\boldsymbol{\mu}_c) + J\delta\mathbf{x}. \tag{1.10}$$

利用 $\mathrm{Cov}(A\mathbf{x}) = A\,\mathrm{Cov}(\mathbf{x})A^T$，得到

$$\boxed{\Sigma_{2D} = JW\Sigma W^T J^T + \epsilon I_2.} \tag{1.11}$$

ϵ 的单位是像素平方，用于屏幕低通或数值正则化。它不是所有路径通用的魔法常数；一些抗锯齿路径还会补偿 opacity，不能只加宽协方差就宣称实现了完整抗锯齿。这个推导只适用于局部线性化，近相机平面、很大的 Gaussian、鱼眼或有畸变的相机需要额外处理。投影裁剪与视场边缘的 Jacobian 限制也属于具体实现，而不是以上理想公式本身。

1.2.2 求逆以后为什么叫 conic?

写成

$$\Sigma_{2D}=\begin{bmatrix}A & B\\ B & D\end{bmatrix},\quad \Delta=AD-B^2>0,\quad \Lambda=\Sigma_{2D}^{-1}=\frac{1}{\Delta}\begin{bmatrix}D & -B\\ -B & A\end{bmatrix}=\begin{bmatrix}a & b\\ b & c\end{bmatrix}. \tag{1.12}$$

实现只需存储 a,b,c 三个独立标量。令像素中心 $\mathbf{p}=(u+0.5,v+0.5)^T$，屏幕 Gaussian 中心为 $\mathbf{m}=\pi(\boldsymbol{\mu}_c)$，偏移定义为 $\mathbf{d}=\mathbf{m}-\mathbf{p}$。它的二次型、半二次型和未截断 alpha 分别为

$$Q(\mathbf{d})=\mathbf{d}^T\Lambda\mathbf{d}=ad_x^2+2bd_xd_y+cd_y^2, \tag{1.13}$$

$$h(\mathbf{d})=\tfrac{1}{2}Q(\mathbf{d}), \tag{1.14}$$

$$\widetilde{\alpha}=o\exp(-h). \tag{1.15}$$

有的源码把 h 命名为 sigma，但它既不是尺度 s，也不是协方差 Σ。式中的交叉项是 $2bd_xd_y$；写成 h 时才变成 bd_xd_y。等值线 $Q=r^2$ 是二次曲线，也就是 conic 在这里的来源。

若 Σ_{2D} 的特征值为 λ_1,λ_2，该等值椭圆沿特征向量的半轴长度为 $r\sqrt{\lambda_1}$ 和 $r\sqrt{\lambda_2}$。所以大协方差意味着宽覆盖、小逆协方差，而不是相反。固定 3σ 外接圆易理解，但会把很多实际没有贡献的 tile 包含进来。

1.2.3 阈值如何变成 tile culling 条件?

令保留阈值为 τ。当 $o\geq\tau$ 时，

$$o\exp(-Q/2)\geq\tau\quad\Longleftrightarrow\quad Q\leq 2\log(o/\tau). \tag{1.16}$$

若 $o<\tau$，即使在中心也达不到阈值；若 $o=\tau$，理论上只剩中心。对于 $\tau=1/255$，右端就是 $2\log(255o)$。这给出了随 opacity 改变的有效椭圆。要剔除整个 tile，必须证明 tile 内的最小 Q 仍超过该界；只检查 tile 中心或四个角都不充分，因为椭圆可能穿过边缘或覆盖内部。保守包围盒容易安全实现，但不等于精确椭圆与矩形相交测试。

投影之后通常先数每个 Gaussian 覆盖多少 tile，通过前缀和分配输出位置，再生成 Gaussian–tile 相交记录，按 tile 和深度排序。一个 Gaussian 可能对应多条记录，所以相交数不是 Gaussian 数，也不是最终有效像素贡献数。本文比较的 gsplat 已经具有 compact-box/sweep 路径，不能把它当成此次新移植的加速收益。

1.3 Alpha compositing：颜色权重来自哪里?

1.3.1 前向递推与背景

对某像素按从近到远的顺序排列实际参与累积的 Gaussian，令 $T_0=1$、$\mathbf{C}_0=\mathbf{0}$。这里 T 是透射率，即尚未被遮挡的比例。

$$w_k=T_{k-1}\alpha_k, \tag{1.17}$$

$$\mathbf{C}_k=\mathbf{C}_{k-1}+w_k\mathbf{c}_k, \tag{1.18}$$

$$T_k=T_{k-1}(1-\alpha_k). \tag{1.19}$$

展开后有

$$T_n=\prod_{k=1}^{n}(1-\alpha_k),\qquad \mathbf{C}_{\text{out}}=\sum_{k=1}^{n}\left[\alpha_k\prod_{j<k}(1-\alpha_j)\right]\mathbf{c}_k+T_n\mathbf{c}_{\text{bg}}. \tag{1.20}$$

累积 alpha 为 $A_{\text{out}}=1-T_n$。权重之和 $\sum_k w_k=1-T_n$，所以连同背景权重恰好为一。这些公式不要求颜色在 $[0,1]$ 中，但若颜色是常规 RGB，就具有直观的混色含义。

交换前后顺序一般会改变颜色，因为后面的颜色要乘上前面留下的透射率。单个像素上的前后递推因此有依赖；大量像素之间却能并行。这正是 tile rasterization 通常让线程对应像素、在线程内部遍历 Gaussian 列表的原因。基于中心深度的排序是 3DGS 的近似处理，不是对所有相交椭球做精确的逐像素体排序。

1.3.2 阈值是渲染语义，不只是性能开关

本次对照路径包含 alpha 上限 0.999、低贡献阈值 $1/255$，以及 $T \le 10^{-4}$ 的停止条件。需要区分三种操作：

1. 将 $\widetilde{\alpha}$ 限制为 $\alpha = \mathsf{min}(0.999, \widetilde{\alpha})$；
2. 若 $h < 0$ 或 alpha 太小，跳过这个候选；
3. 先算 $T_{\mathsf{next}} = T(1-\alpha)$，若满足终止条件则退出。

在本文对照的 exclusive termination 中，第 3 步触发退出的那个 Gaussian 也不累积颜色，T 保持在上一次已接受贡献之后的值。若改为先累积再退出，就变成 inclusive termination，不能声称与原实现逐 bit 等价。理论上正定 conic 保证 $h \ge 0$；源码检查负值是防御性处理，不应把负值当作正常 Gaussian 的特征。

阈值还意味着函数分段光滑。一般 backward 对当前选中的排序、贡献集合及分支求导，不对离散排序或“刚好跨过阈值”这一事件求通常意义的连续导数。以下 VJP 推导针对已接受、未饱和且不在阈值边界上的贡献。

1.3.3 手算一个有背景的例子

取

$$\Sigma_{2D} = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}, \quad \Lambda = \begin{bmatrix} 1/4 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{d} = (2,0)^T, \quad o_1 = 0.8. \tag{1.21}$$

于是 $Q = 1$、$h = 0.5$，第一层 alpha 为

$$\alpha_1 = 0.8\,\mathsf{exp}(-0.5) \approx 0.485224528. \tag{1.22}$$

令第一层为红色 $(1,0,0)$，第二层为绿色 $(0,1,0)$，第二层在该像素的 alpha 为 $\alpha_2 = 0.25$，背景为蓝色 $(0,0,1)$。逐步计算得到

$$w_1 = 0.485224528, \qquad T_1 = 0.514775472, \tag{1.23}$$

$$w_2 = T_1 \times 0.25 = 0.128693868, \qquad T_2 = T_1 \times 0.75 = 0.386081604, \tag{1.24}$$

$$\mathbf{C}_{\mathsf{out}} = (0.485224528,\ 0.128693868,\ 0.386081604), \qquad A_{\mathsf{out}} = 0.613918396. \tag{1.25}$$

RGB 三个分量在此恰好是红、绿、蓝三层的权重，和为一。若把第二层绿色放到前面，红色权重变为 $0.75\alpha_1$，绿色权重变为 0.25；总透射率不变，但颜色改变。这是遮挡顺序不能随意重排的最小示例。

1.4 Reverse compositing 与 VJP：为什么反向要倒着走?

1.4.1 VJP 不是构造一个巨大 Jacobian

若 $\mathbf{y} = f(\mathbf{x})$，损失传来的上游梯度为 $\overline{\mathbf{y}} = \partial L/\partial \mathbf{y}$，反向需要的是 $\overline{\mathbf{x}} = J_f^T \overline{\mathbf{y}}$。这称为 vector–Jacobian product 的反向传播形式；实际计算不必显式保存图像所有像素对全部 Gaussian 参数的巨型 Jacobian。一个线程先算本像素的局部贡献，再将共享 Gaussian 的梯度归并即可。

用上横线表示伴随量。忽略背景时，一层状态更新的局部反向为

$$\overline{\mathbf{c}}_k = T_{k-1}\alpha_k\overline{\mathbf{C}}_k, \tag{1.26}$$

$$\overline{\alpha}_k = T_{k-1}\left(\overline{\mathbf{C}}_k \cdot \mathbf{c}_k - \overline{T}_k\right), \tag{1.27}$$

$$\overline{T}_{k-1} = \alpha_k\left(\overline{\mathbf{C}}_k \cdot \mathbf{c}_k\right) + (1-\alpha_k)\overline{T}_k, \tag{1.28}$$

$$\overline{\mathbf{C}}_{k-1} = \overline{\mathbf{C}}_k. \tag{1.29}$$

这些等式直接来自对 $\mathbf{C}_k = \mathbf{C}_{k-1} + T_{k-1}\alpha_k\mathbf{c}_k$ 和 $T_k = T_{k-1}(1-\alpha_k)$ 分别求导。opacity 增大时，一方面增加本层颜色，另一方面压暗全部后景；$\overline{\alpha}_k$ 中的两个项正好对应这两种效果。

若最终输出含背景和累积 alpha，初始化应为

$$\overline{\mathbf{C}}_n = \overline{\mathbf{C}}_{\text{out}}, \qquad \overline{T}_n = \overline{\mathbf{C}}_{\text{out}} \cdot \mathbf{c}_{\text{bg}} - \overline{A}_{\text{out}}, \tag{1.30}$$

这里假定没有额外直接输出 T_n；若有，还需加上它的上游梯度。不能把背景项漏掉，否则 opacity 梯度即使在颜色图像看似正常时也可能错误。期望深度一类含归一化的输出还要先对归一化做 VJP，不能直接套用 RGB 的上游梯度。

1.4.2 恢复透射率与尾部裁剪

从最后一个已接受贡献开始，有

$$T_{k-1} = \frac{T_k}{1-\alpha_k}. \tag{1.31}$$

因此可以保存最终透射率和最后贡献索引，再反向重算 alpha，而不为每个像素保存完整的所有中间状态。alpha cap 保证分母非零，但除法恢复并不等于撤销一次浮点乘法，仍会有舍入误差。被 cutoff 跳过的候选不能误当成已接受贡献来恢复 T；exclusive termination 中触发退出的候选同样不在这个递推序列内。

同一 tile 的不同像素可能在不同深度停止。若每个像素保存最后实际贡献的相交索引 ℓ_p，则 tile 的共同反向上界可以取 $\max_p \ell_p$。其后的候选对 tile 中所有像素都没有参与前向，反向便可一起省略。这就是完全遮挡尾部裁剪的数学依据：它删除的是已证明无贡献的公共尾部，并不意味着可以任意删除中间候选、重排 ID，或更改停止阈值。

1.4.3 从 alpha 梯度回到 conic 和中心

在未触及 cap 的光滑区域，设 $\overline{\alpha} = \partial L/\partial\alpha$，则

$$\overline{o} = \overline{\alpha}\exp(-h), \tag{1.32}$$

$$\overline{h} = -\overline{\alpha}\alpha, \tag{1.33}$$

$$\overline{a} = \tfrac{1}{2}\overline{h}\,d_x^2, \qquad \overline{b} = \overline{h}\,d_x d_y, \qquad \overline{c} = \tfrac{1}{2}\overline{h}\,d_y^2, \tag{1.34}$$

$$\overline{\mathbf{m}} = \overline{h}\,\Lambda\mathbf{d}. \tag{1.35}$$

最后一式使用了本章的 $\mathbf{d} = \mathbf{m} - \mathbf{p}$；若定义反过来，中间符号也要跟着改变。这里 b 表示对称矩阵两个位置共享的一个存储参数，所以其导数已经包含两处贡献。如果将矩阵每个元素视为独立变量，不能再把这个打包梯度原样塞入两个非对角位置。

数学上的 min 在上限以上导数为零，在拐点没有唯一通常导数。实际 CUDA backward 是否严格使用该分段导数、采用何种边界约定，应以待比较版本为准；不能仅从前向公式推定所有历史实现的梯度规则。本节公式只用于未饱和贡献，不能替代对 cap、cutoff 和终止分支的源码回归测试。

协方差求逆的微分满足

$$d\Lambda = -\Lambda\,(d\Sigma_{2D})\,\Lambda. \tag{1.36}$$

其后还要反传到 J, W, Σ，再到尺度和四元数。由于 J 依赖中心位置，三维中心梯度不仅有屏幕中心那一条路径，还有屏幕协方差随投影 Jacobian 改变的路径。这说明“screen-space gradient 对齐”是重要的局部证据，但不是完整参数梯度验证的替代品。

1.4.4 用手算例子检验反向直觉

仍取前述红、绿、蓝例子，令损失只是最终绿色分量 $L = C_{\text{out},G}$。于是 $L = (1-\alpha_1)\alpha_2$，直接得到

$$\frac{\partial L}{\partial \alpha_1} = -\alpha_2 = -0.25, \qquad \frac{\partial L}{\partial \alpha_2} = 1 - \alpha_1 \approx 0.514775472. \tag{1.37}$$

第一层红色自身没有绿色，增加其 opacity 却会挡住后面的绿色，故梯度为负。若只写“alpha 增大则本层贡献增大”，而没有反向的后景项，就无法得到这个负号。

1.5 可运行的最小 Python 数学实验

以下代码只依赖 Python 标准库，不需要 PyTorch、NumPy 或 GPU。它演示未触发 cap、cutoff 和 early termination 的两层情形，不是实际 gsplat 的浮点精度或性能基准。可将代码复制到脚本中运行；在本仓库运行时应使用已有的 `@venv/bin/python`，没有该环境时不要为本章安装依赖。

```
1  import math
2
3  def gaussian_alpha(dx, dy, a, b, c, opacity):
4      h = 0.5 * (a * dx * dx + 2 * b * dx * dy + c * dy * dy)
5      return opacity * math.exp(-h)
6
7  a1 = gaussian_alpha(2.0, 0.0, 0.25, 0.0, 1.0, 0.8)
8  a2 = 0.25
9  colors = [(1.0, 0.0, 0.0), (0.0, 1.0, 0.0)]
10 background = (0.0, 0.0, 1.0)
11 C, T = [0.0, 0.0, 0.0], 1.0
12 for alpha, color in zip((a1, a2), colors):
13     weight = T * alpha
14     C = [x + weight * y for x, y in zip(C, color)]
15     T *= 1.0 - alpha
16 C = [x + T * y for x, y in zip(C, background)]
17 print("alpha1, RGB, A:", a1, C, 1.0 - T)
18
19 def green_loss(alpha1):
20     return (1.0 - alpha1) * a2
21
22 eps = 1e-6
23 finite_diff = (green_loss(a1 + eps) - green_loss(a1 - eps)) / (2 * eps)
24 analytic = -a2
25 assert math.isclose(sum(C), 1.0, abs_tol=1e-12)
26 assert math.isclose(finite_diff, analytic, rel_tol=1e-8, abs_tol=1e-9)
27 print("d_green/d_alpha1:", analytic, finite_diff)
```

逐行理解如下，行号包括空行：

第 1 行 导入指数函数；没有调用随机数或外部渲染库。

第 3 行 函数参数是像素偏移、打包 conic 和已激活 opacity。

第 4 行 计算 $h = Q/2$；注意交叉项中的二倍因子。

第 5 行 返回未截断 alpha，不含概率密度的归一化常数。

第 7 行 代入手算例子的第一层 Gaussian。

第 8 行 直接指定第二层在这个像素的 alpha，不假装重新构造三维场景。

第 9–10 行 分别定义前后层 RGB 和背景 RGB。

第 11 行 初始化累计颜色与透射率，不能把初始 T 写成零。

第 12 行 按固定从近到远顺序遍历两层。

第 13 行 用更新前的 T 计算当前层权重。

第 14 行 对三个颜色通道累加本层贡献。

第 15 行 当前层累加后，更新留给后景的透射率。

第 16 行 将剩余透射率分配给背景。

第 17 行 打印第一层 alpha、最终颜色和总 alpha，可与手算值对照。

第 19–20 行 把绿色通道写成独立的标量损失函数。

第 22 行 指定双精度中心差分步长；不是 CUDA float32 的推荐通用步长。

第 23 行 用两次函数求值近似 $\partial L/\partial \alpha_1$。

第 24 行 写出解析梯度，它是负数。

第 25 行 检查本例三个纯色权重之和为一；一般 RGB 不满足这个断言。

第 26 行 检查有限差分与解析导数相符。

第 27 行 打印二者，预期都接近 -0.25。

空行只是区分计算阶段。该实验验证了公式和符号，不验证并行 atomic 顺序、阈值边界、相机模型、SH 或真实 CUDA backward 的正确性。

1.6 把数学放到 GPU：线程、存储和同步

1.6.1 Block、warp 和 lane 的层级

一次 kernel launch 产生一个 grid，grid 包含多个 thread block。一个 block 被安排在某个流式多处理器 SM 上执行；一个 SM 可以同时驻留多个 block。CUDA 程序不能假定不同 block 的执行顺序，也不能用普通 block barrier 同步整个 grid。

NVIDIA GPU 的 warp 通常包含 32 个线程，lane 是线程在 warp 内的编号 $0, \dots, 31$。二维 block 的线性编号是

$$t = \texttt{threadIdx.x} + \texttt{blockDim.x}\,\texttt{threadIdx.y}, \qquad \text{lane} = t \bmod 32. \tag{1.38}$$

因此 16×16 的 block 有 256 个线程，即 8 个完整 warp；4×4 只有 16 个线程，是一个不完整 warp。这不是纯粹的性能细节：使用 full-mask shuffle 的优化若假定所有 32 个 lane 都存在，就不能无条件覆盖后者。本文实验最终为非整 warp 大小的 block 保留原 reduction 回退路径。

典型 tile kernel 让一个 block 处理一个 tile，每线程负责一个像素。这里的“负责”是逻辑映射，不代表像素一定有贡献，也不代表每个 lane 都做一样多的有效计算。边缘 tile 中的越界像素、提前被遮挡的像素仍可能需要参与 block 级同步。

1.6.2 寄存器、shared memory、L2 和显存

寄存器 保存线程私有的 T、颜色、偏移和梯度临时量。访问便宜，但数量有限。每线程寄存器越多，可能允许同时驻留的 warp 越少；编译器 spill 到 local memory 后，还会产生经过缓存的设备内存访问。因此“把所有数据都提前读进来”未必更快。

Shared memory block 内共享、显式管理的片上存储。多个像素会读取同一批 Gaussian，可协作加载后复用，减少重复全局读取。同步、bank conflict 和 shared 容量也有成本，使用 shared 并不自动带来加速。

L2 cache GPU 上多个 SM 共享的缓存层。它能复用 Gaussian 属性、相交表和部分梯度访问，但不能代替程序所需的同步。是否命中由工作集和访问顺序决定，不由“这个数组常被访问”直接保证。

显存 保存全部参数、相交记录、输出和梯度，容量大但访问延迟与带宽成本高。相邻 lane 读取相邻地址通常更容易形成合并访问。反向随机写回、重复 ID 和大相交表则可能增加流量或争用。

RTX 4090 属于 Ada 架构、计算能力 8.9，具有 128 个 SM 和 72 MB L2。其显存为 24 GB，标称显存带宽约为 1008 GB/s；实际可达带宽取决于访问模式。以本次 $20{,}789{,}814$ 个 Gaussian 为例，仅每个 Gaussian 一个 float32 标量就约占 83.2 MB，已经大于这块 GPU 的 L2。这不等于每次访问都会 miss：可见子集、tile 内复用和相机视角仍然很重要。但它说明不能把整场景参数常驻 L2 当作默认前提，也不能直接复制 A100 的性能结论。

1.6.3 为什么 backward 需要 atomic add?

前向每个像素通常只写自己的输出，不存在多个线程争写同一个颜色位置的问题。反向则不同：一个 Gaussian 影响很多像素，多个像素线程会同时贡献 $\overline{\mathbf{m}}_i, \overline{\Lambda}_i, \overline{o}_i, \overline{\mathbf{c}}_i$。普通的“读取、相加、写回”会丢失并发更新，需要 atomic add 或其他正确的归并机制。

若同一个 warp 中的 lane 当前处理同一个 Gaussian，可以先做 warp reduction，再由少数 lane 发出 atomic，将最多 32 份更新压缩成较少的更新。关键前提是归并键确实相同；任意 32 个线程的梯度不能因为相邻就直接相加。reduce-scatter 进一步把不同梯度分量的最终和分散到不同 lane，避免全部分量由 lane 0 串行完成。它改变的是归并和发射方式，不是梯度的数学定义。

atomic 保证单次更新不会互相覆盖，但通常不保证来自不同 warp 的加法顺序。float32 不满足实数的结合律：

$$\mathrm{fl}(\mathrm{fl}(a+b)+c) \neq \mathrm{fl}(a+\mathrm{fl}(b+c)). \tag{1.39}$$

因此减少 atomic 或改 reduction 树，即使前向完全不变，也可能改变梯度的末位。不能把所有误差都归因于 atomic 噪声；仍要分别排除漏梯度、错误 mask 和越界。本次结果中，单次 full-path quaternion 相对 L2 约 1.63×10^{-4}，没有通过严格 10^{-4} 门槛；重复平均的结果更接近，不能抹去这条限制。

1.7 CUDA 教学伪代码：协作加载与像素递推

下面是**概念 CUDA 伪代码，不可直接编译，也不是 gsplat 源码替代品**。假定一维 block 的线程数为 `B`，每个 tile 有 `B` 个像素，每批也最多加载 `B` 个候选。`G`、`Candidate`、tile 范围及颜色运算都是解释用抽象。所有 Gaussian 属性均假定有限；为突出同步和 compositing，没有展开排序及反向。

```
__shared__ Candidate batch[B];
int t = threadIdx.x;
Pixel p = pixel_for(blockIdx.x, t);
bool inside = pixel_inside_image(p);
bool done = !inside;
float T = 1.0f;
Color C = zero_color();
for (int base = begin; base < end; base += B) {
    int count = min(B, end - base);
    if (t < count) batch[t] = G[sorted_ids[base + t]];
    __syncthreads();
    if (!done) {
        for (int j = 0; j < count; ++j) {
            Candidate g = batch[j];
            Vec2 d = g.mean - pixel_center(p);
            float h = 0.5f * (g.a*d.x*d.x + g.c*d.y*d.y)
                      + g.b*d.x*d.y;
            float alpha = min(0.999f, g.opacity * expf(-h));
            if (h < 0.0f || alpha < 1.0f/255.0f) continue;
```

```
20              float next_T = T * (1.0f - alpha);
21              if (next_T <= 1e-4f) { done = true; break; }
22              C += (T * alpha) * g.color;
23              T = next_T;
24          }
25      }
26      __syncthreads();
27      if (__syncthreads_count(!done) == 0) break;
28  }
29  if (inside) store(p, C + T * background, 1.0f - T);
```

逐行解释：

第 1 行 为整个 block 声明共享候选缓冲，不是每线程一份。

第 2 行 取得一维 block 中的线程编号，二维情形需先线性化。

第 3 行 将 block 和线程映射到 tile 内像素。

第 4 行 判断边缘 tile 的像素是否位于图像内。

第 5 行 越界线程不参与像素数学，但稍后仍参与协作和同步。

第 6–7 行 在寄存器中初始化透射率和累计颜色。

第 8 行 分批遍历已经按深度排序的相交记录范围。

第 9 行 最后一批可能不足一个 block。

第 10 行 各线程各加载一个候选；即使像素已经 done，线程仍可承担加载。

第 11 行 等全部共享写入完成后才能读取，所有线程都必须到达。

第 12 行 已终止的像素跳过后续数学，而不是从 kernel 提前返回。

第 13 行 每个活跃像素按相同候选顺序处理当前批。

第 14 行 读取共享属性；是否被编译器放进寄存器取决于实际代码和编译结果。

第 15 行 按前文定义计算中心减像素中心的偏移。

第 16–17 行 计算半二次型，交叉项在此只乘一次 b。

第 18 行 计算并限制 alpha；这里的 expf 不承诺与源码近似指令逐 bit 相同。

第 19 行 丢弃无效或过小贡献；这是可能出现 warp 分歧的位置。

第 20 行 预计算若接受当前层后的透射率。

第 21 行 保留 exclusive termination：当前候选不进入颜色累积。

第 22 行 使用旧 T 计算颜色权重。

第 23 行 仅在该贡献被接受后更新 T。

第 24–25 行 结束候选循环和活跃像素分支，重新汇合到公共同步位置。

第 26 行 等所有线程读完共享缓冲，防止下一批覆盖尚在使用的数据。

第 27 行 全 block 共同判断是否都已结束；此 collective 不能只让活跃线程调用。

第 28 行 当前批结束；若还有活跃像素则协作加载下一批。

第 29 行 仅对图像内像素写回颜色及总 alpha，并补上背景。

这个框架故意保留两处清晰的 barrier，便于检查生命周期，而不是追求最少同步次数。实际 kernel 可能采用不同批大小、向量化属性、异步加载和专门的 warp primitive。优化前必须先说明候选由谁加载、何时可读、何时可以覆盖；仅仅把 `continue` 改成乘零，不会自动满足这些契约。

1.8 SIMT 分歧：少一个分支为什么不一定更快?

SIMT 让一个 warp 的多个 lane 执行同一条指令流，但每个 lane 有自己的数据。如果一部分 lane 进入分支、另一部分不进入，硬件可能用不同执行掩码先后执行路径。这不等于启动两组新线程；代价主要是部分 lane 暂时不能有效利用指令槽。如果整个 warp 都跳过一大段昂贵计算，分支反而很有价值。

对于 `alpha < cutoff`，可以设想把无效 lane 的 alpha 置零，统一继续计算。这可能降低控制流开销，却也可能执行原本能跳过的指数、加载和梯度计算，增加寄存器活跃区间，并让更多无效值进入计算。特别是 IEEE 浮点中 $0 \times \infty$ 和 $0 \times \mathrm{NaN}$ 都不是安全的零。所以“数学上乘零无贡献”不足以证明 branchless backward 安全。

本次保留的是受限的、安全的窄通道 alpha predication：无效或相应饱和状态的 lane 仍需按实现规则阻止危险的 geometry、opacity、absgrad 和颜色缓冲更新。这与“所有分支全部删除”不是一回事。也不能只看 CUDA 源码判断最终有没有分支；编译器可能将短条件转换为 predication。真实性能应结合生成代码、活跃 lane 比例和 profiler，而不是以源文件中的 if 数量衡量。

1.9 如何读懂 RTX 4090 上的计时：roofline 与 Amdahl

1.9.1 Roofline 是上界模型，不是自动诊断

设某段计算完成 F 次浮点运算，实际从所研究存储层传输 M 字节，算术强度定义为 $I = F/M$。最简单的 roofline 给出吞吐量上界

$$P \le \min(P_{\text{peak}}, B_{\text{mem}} I), \qquad t \gtrsim \max\left(\frac{F}{P_{\text{peak}}}, \frac{M}{B_{\text{mem}}}\right). \tag{1.40}$$

其含义是：运算量很少而搬运量很大时，单纯减少几次乘加不一定明显加速；反过来，如果已经受计算吞吐限制，减少显存字节也未必立即见效。使用显存带宽时，M 应是相应层级的实际流量，而不是简单把源代码 load 次数乘以四。L2 和 shared 复用会改变它，多级存储可以建立各自的 roofline。

例如一次简化计算若做 32 FLOP、产生 32 字节显存流量，则 $I = 1$ FLOP/byte。使用约 1008 GB/s 的理论带宽，其带宽上界约为 1.008 TFLOP/s，远低于 RTX 4090 约 82.6 TFLOP/s 的标称 FP32 峰值。这只是假设流量下的例子，不是对本次 kernel 的实测分类。FLOP 统计通常把一个 FMA 计作两次运算；指数、排序、原子操作和同步不能简单按相同成本折算。

实际 raster backward 还可能受同地址 atomic 争用、指令延迟、寄存器压力、活跃 warp 数不足或 warp 分歧限制。因此 warp reduce-scatter 的收益不能只用“减少了多少 FLOP”解释。需要 profiler 区分 DRAM/L2 流量、atomic 活动、stall、occupancy 和指令发射瓶颈。高 occupancy 是隐藏延迟的手段，而不是必须最大化的最终指标。

1.9.2 Amdahl：raster 加速不能直接当成完整渲染加速

设完整流程中可优化部分耗时占比为 f，该部分加速 s 倍，其余成本不变，则

$$S_{\text{full}} = \frac{1}{(1-f) + f/s}. \tag{1.41}$$

本次 RTX 4090、1280×720、全量约 2080 万 Gaussian 的 RGB camera 0 记录中，raster 为 $23.319 \to 17.444$ ms，即约 1.337 倍; 完整路径为 $69.148 \to 62.851$ ms，即约 1.100 倍。若粗略取 $f = 23.319/69.148 \approx 0.337$，Amdahl 预测完整耗时约为

$$69.148 - 23.319 + 17.444 = 63.273 \text{ ms}, \tag{1.42}$$

相应加速约 1.093 倍，量级与实测一致。两者无需严格相等：独立计时存在波动，缓存、其他阶段和测量边界也可能变化。不能把不同测量中的中位数强行当成同一次运行的可加分项。

camera 2552 的 raster 记录为 $3.269 \rightarrow 1.984$ ms，约 1.648 倍，完整路径却只有 $40.360 \rightarrow 39.728$ ms，约 1.016 倍。这不是矛盾，而是 raster 已经只占很小一部分；投影、相交生成、排序和其他后向阶段仍然存在。即使把占比 f 的部分彻底变成零成本，完整加速上限也只有 $1/(1-f)$。

1.9.3 异步执行与可比较的实验条件

CUDA launch 对主机通常是异步的，用普通 CPU 时钟只包围一次 launch，很可能测到的是提交时间而非 GPU 完成时间。常见方法是在同一 stream 上放置 CUDA events，等待结束 event 后读取间隔。若使用 CPU 墙钟，则必须在合适边界同步，并明确是否把同步和主机工作计入。跨 stream 的依赖需要显式事件或其他同步，不能默认事件覆盖所有异步工作。

对本次比较，至少要固定以下条件：

1. 相机、分辨率、点数、SH 阶数和相交生成方式一致；
2. RGB、RGB+ED、absgrad、packed/unpacked 等路径不能混作同一个指标；
3. 预热后重复测量，避免将 JIT 编译、扩展加载和首次内存分配当成稳定 kernel 时间；
4. 说明 raster-only 与 full-render 的边界，尤其是否包含 backward 和梯度清零；
5. 固定二进制与版本，并记录 GPU 状态，尽量控制其他 GPU 负载、温度和频率漂移。

性能和正确性应分开检查：图像/alpha 一致、局部 screen-space VJP 一致、原始参数 VJP 一致，以及训练长期行为一致，是不同层级的结论。sanitize 检查也有独立价值：没有图像差异不代表没有越界或错误同步。

1.10 把本章公式映射回优化问题

现在可以沿着同一条逻辑链阅读博客和 gsplat 对照：

1. 用 log-scale、四元数构造三维形状，确认传给接口的是原始参数还是激活值。
2. 用局部投影得到屏幕 conic，再利用 Q 与 opacity 阈值估计覆盖范围。
3. 将覆盖关系转换成有序相交列表；这一阶段决定后续要处理多少候选。
4. 像素线程按 $T\alpha$ 递推，严格保留阈值和 exclusive termination 的语义。
5. 反向从最后有效贡献恢复透射率，用 VJP 将颜色损失转成 Gaussian 属性梯度。
6. 在 lane、warp 与 block 层面安全归并，减少无用尾部遍历和有争用的原子更新。
7. 最后用完整流水线计时和分层误差验证，判断收益是否足以覆盖复杂度。

本次真正保留的 backward 优化是 warp reduce-scatter、完全遮挡尾部裁剪，以及安全的窄通道 alpha predication。这些操作分别作用于归并方式、候选遍历范围和执行掩码；并不更改 Gaussian 表示、投影方程或目标损失。但数学目标相同不等于执行细节天然等价：四元数约定、conic 非对角系数、浮点累加顺序、无效 lane 和同步边界，都必须纳入验证。理解这一点，才能把“少做一些工作”的优化直觉，变成在 RTX 4090 上可解释、可复核的工程结果。

Chapter 2

原文章逐点解读：S00–S06

2.1 阅读地图：把文章当作一条可验证的优化链

George Kopanas 的文章 What the 3DGS Literature Left on the Table 不是一份 CUDA 技巧清单，而是一堂关于**瓶颈迁移**的课程：先消除不必要的相交对，再降低原子操作和归约成本，然后处理寄存器、无用反向工作、分歧以及执行流水线。前一个步骤改变了后一个步骤成立的条件。例如，没有精确的子块筛选，把逐像素分支全部改成无分支计算，可能只是让更多无效像素消耗算力。

本章依次讲解 s00 至 s06。每一步都区分三件事：文章的设计、本仓库所用 gsplat 分支原来具备的能力，以及这次移植实验实际保留的内容。代码片段是解释依赖关系的教学伪代码，不是可以直接替换通用内核的补丁；省略的边界、同步和张量布局检查不能在生产代码中省略。

先划清数字的适用边界。 文章使用一张 NVIDIA A100-SXM4-40GB，在九个 Mip-NeRF 360 场景的 7k、30k 检查点上测量。其标题中的 $7.30\times$（30k）和 $8.38\times$（7k）对应最终 BF16 SH 版本的前向加反向渲染流水线，与 Inria 参考实现比较；它们不是任意训练器的优化器、损失、数据加载与增密循环的整体倍率。文章最终 FP32 SH 版本在相应集合上的数字也不同：分别为 $6.54\times$ 和 $7.55\times$。不能把 BF16 的标题归到纯 FP32 版本上。

本任务使用 RTX 4090、20,789,814 个高斯、一阶 SH、1280×720 图像，对比的是已经具备部分优化的 gsplat 分支。硬件、场景、初始实现、SH 阶数和接口契约都不相同。因此**文章的 A100 计数器、启动参数和标题倍率均不可直接迁移**。本章中有关本任务的结论以 `benchmarks/blog_eval/REPORT.md` 的最终重建记录为准，不把较早的探索性数字拼接成一个不存在的最终结果。

2.1.1 先建立一个关于成本的模型

记总高斯数为 N，当前相机可见的高斯数为 M，发出的 tile–Gaussian 相交对数为 P，实际执行的像素–Gaussian 混合次数为 E。这四个数决定了不同阶段的主要工作量：

$$t_{\text{step}} \approx t_{\text{project}}(N) + t_{\text{SH}}(M) + t_{\text{bin/sort}}(P, M) + t_{\text{raster}}(E) + t_{\text{VJP}}(N, M). \tag{2.1}$$

这里 VJP 指向量–雅可比乘积，即反向传播中的链式法则计算。该式不是拟合出的性能公式，而是一张诊断地图：如果排序占主导，优化归约并不会让排序自动变快；如果反向预处理仍扫描全部 N，减少像素混合次数也不会消除所有大场景开销。

假设某阶段占原总时间的比例为 f，该阶段获得倍率 s，则整个调用最多变为

$$S_{\text{total}} = \frac{1}{(1-f) + f/s}. \tag{2.2}$$

这解释了为什么本任务可以看到明显的 raster 加速，却只有较小的完整渲染收益。还要注意，CUDA-event 的内核阶段之和与同步的端到端延迟不是同一个统计量：启动、框架调度、等待和内存初始化可以出现在后者中。文章的一视图阶段图、九场景均值，以及本任务三视图测量，不能混在同一张倍率表里。

2.2 s00：忠实基线比一个慢基线更重要

2.2.1 五阶段流水线和不可随意改变的语义

文章的 s00_baseline 是在统一测试框架中忠实重写 Inria CUDA 光栅器，而不是故意制造一个低效版本。后续包逐步增加优化，使每一段差异可以单独归因。文章对 Inria 的倍率仍以实测的 Inria 为分母，并不偷偷换成更慢的重写版本。

一个高斯的世界空间参数为位置、尺度、旋转、不透明度和 SH 系数。投影之后，光栅器主要使用二维中心 $\boldsymbol{\mu}'$、逆协方差

$$\Lambda = \begin{bmatrix} a & b \\ b & c \end{bmatrix}, \qquad q(\boldsymbol{d}) = ad_x^2 + 2bd_xd_y + cd_y^2, \qquad \boldsymbol{d} = \boldsymbol{p} - \boldsymbol{\mu}'. \tag{2.3}$$

对正定的 Λ，$q \geq 0$。高斯权重和混合不透明度为

$$G = \exp(-q/2), \qquad \alpha_{\text{raw}} = oG, \qquad \alpha = \min(\alpha_{\max}, \alpha_{\text{raw}}). \tag{2.4}$$

这里只把 o 定义为已经激活后的不透明度；若保存的是 logit，调用前需要 sigmoid。同理，保存的对数尺度必须先取指数。把训练文件中的原始参数直接传给要求物理尺度的接口，虽然可能仍能渲染，却已经不是同一个场景。本任务早期一次探索就犯过对数尺度未激活的错误，那些时间和图像比较均已作废。

按深度从前到后，颜色与透射率的递推为

$$\boldsymbol{C}_k = \boldsymbol{C}_{k-1} + T_{k-1}\alpha_k\boldsymbol{c}_k, \tag{2.5}$$

$$T_k = T_{k-1}(1-\alpha_k), \qquad T_0 = 1, \tag{2.6}$$

$$\boldsymbol{I} = \boldsymbol{C}_K + T_K\boldsymbol{c}_{\text{bg}}. \tag{2.7}$$

低于 cutoff 的 splat 不参与混合；像素达到透射率终止条件后停止。特别要区分：检测终止的是更新前还是更新后的 T，触发终止的 splat 是先混合还是先拒绝，以及停止索引采用包含还是不包含端点。这些是基线语义，不是实现小节。

```
// Semantic skeleton: keep the reference's precise stop ordering.
project_and_shade();
count_pairs_then_prefix_scan();
emit_and_depth_sort_per_tile();
for (splat : front_to_back_list) {
    alpha = evaluate_with_reference_cutoff_and_cap(splat, pixel);
    if (!reference_accepts(alpha, T)) continue;
    C += (T * alpha) * color;
    T *= (1.0f - alpha);
    record_reference_last_index();
}
// Backward replays the accepted prefix in reverse order.
for (splat : reverse_accepted_prefix) {
    T_before = T_after / (1.0f - alpha);
    accumulate_screen_space_vjp();
}
apply_projection_covariance_and_SH_vjp();
```

2.2.2 反向为什么昂贵

一个高斯会影响多个像素、多个 warp、甚至多个 tile。屏幕空间梯度通常包含中心 2 项、conic 3 项、不透明度 1 项、RGB 3 项，共 9 项。若每个像素直接向全局数组执行 9 次原子加，许多线程会竞争同一高斯地址。反向还需要逆序重建透射率：

$$T_{k-1} = \frac{T_k}{1-\alpha_k}. \tag{2.8}$$

因此 alpha cap 不只控制图像，也保证分母不为零；改变 cap 会同时改变渲染和梯度。文章示意路径使用 0.99，本任务必须保留 gsplat 的 0.999。两者不能因为 “都很接近 1” 就互换。

预期瓶颈与风险。 忠实基线常见的瓶颈是宽松分箱产生过多 P、逐像素原子操作以及反向重放。但是基线还承担可验证性职责：相等深度顺序、像素中心、背景梯度、clamp 导数、空输入和异步流都必须固定。PSNR 相同不意味着参数梯度相同，图像误差很小也不能单独证明反向正确。

本分支与本任务。 本任务没有把 gsplat 改写成文章的 s00，而是以固定提交 `ffe2db3` 为自身基线。最终比较还固定参数激活、四元数归一化和 compact-box 配置。文章对 dense reference 的阈值提供了方法论，但不能代替本分支的差分测试。

2.3 s01：先把五类已有文献优化讲透

文章把 DISTWAR、Taming 3DGS、StopThePop、FlashGS、Speedy-Splat、Splatshop、Faster-GS、HiGS、gsplat、LichtFeld Studio 等工作中的技术综合起来。这一步的价值在于消除基线遗漏，不应将其全部算作本任务的新发明。

2.3.1 第一类：warp butterfly 归约，先少写再谈快算

同一个 warp 的 32 个像素线程在同一次迭代处理同一个高斯，设线程 ℓ 对梯度分量 j 的贡献为 $g_{\ell j}$。需要写入的是

$$G_j = \sum_{\ell=0}^{31} g_{\ell j}, \qquad j = 0, \dots, 8. \tag{2.9}$$

先在寄存器之间归约，最后仅由 lane 0 写每一项，可把同一 warp 的原子更新数最多降低 32 倍。shuffle 是 lane 间寄存器交换，不经过共享内存。

```
// Every lane must participate; rejected pixels supply exact zero.
for (int j = 0; j < 9; ++j) {
    float sum = grad[j];
    for (int d = 16; d != 0; d >>= 1)
        sum += __shfl_xor_sync(0xffffffffu, sum, d);
    if (lane == 0) atomicAdd(dst + j, sum);
}
```

5 次交换之后每个线程都有总和，但其中 31 份会被丢弃。9 个分量需要 $9 \times 5 = 45$ 条 shuffle，lane 0 还要发出 9 条原子指令。这里的 “一条指令” 是 warp 级发射，不是一个地址上的交易量；减少指令数不等于消除跨 tile 对同一高斯的竞争。

瓶颈、风险和状态。 该方法针对原子压力，随后瓶颈会迁移到 shuffle 和指令发射。必须保证归约成员都处理同一个高斯，且不能让部分 lane 提前退出而仍使用完整掩码。本 fork 原来已有逐分量全 warp 归约与 lane-0 原子写回；本任务的新内容是下一节的 reduce-scatter，不是再次发明 warp 聚合。

2.3.2 第二类：不透明度感知的精确行跨度

原始 3σ 外接圆再包成正方形，忽略了椭圆各向异性，也忽略了低不透明度高斯的有效面积。令 cutoff 为 $\tau = 1/255$，只要 $o \geq \tau$，有效区域满足

$$o \exp(-q/2) \geq \tau \quad \Longleftrightarrow \quad q \leq \rho^2, \qquad \rho^2 = 2 \ln(o/\tau). \tag{2.10}$$

若 $o < \tau$，整个高斯都不能通过该 cutoff；若恰好相等，边界退化情形仍需按比较符号处理。令 $D = ac - b^2 > 0$，椭圆的紧 AABB 半宽为

$$x_{\max} = \rho\sqrt{c/D}, \qquad y_{\max} = \rho\sqrt{a/D}. \tag{2.11}$$

AABB 已经优于最大特征值圆，但倾斜椭圆的四角仍会产生无效 tile 测试。

对固定相对纵坐标 y，解二次方程可以得到横向边界

$$x_{\pm}(y) = -\frac{b}{a}y \pm \sqrt{\frac{\rho^2}{a} - \frac{D}{a^2}y^2}. \tag{2.12}$$

真正的 tile 行不是一条水平线，而是一个纵向带状区间 $[y_0, y_1]$。该行跨度要取区间中 x_- 的最小值与 x_+ 的最大值：检查区间端点以及落在区间内的驻点，再将连续坐标映射成离散 tile 索引。只在行中心代入公式会漏掉擦过上下边缘的椭圆；那不是精确行跨度。

```
if (opacity < cutoff) return no_pairs;
rho2 = 2 * log(opacity / cutoff);
for (int ty = first_row; ty < end_row; ++ty) {
    Strip strip = reference_tile_strip(ty);
    Bounds x = extrema_of_ellipse_over_strip(conic, rho2, strip);
    Span span = conservative_discrete_tile_span(x, image_bounds);
    for (int tx = span.begin; tx < span.end; ++tx)
        emit(tx, ty, gaussian_id);
}
```

瓶颈与风险。 这是减少 P 的算法性优化，同时减轻发射、排序、共享 staging 与像素测试。它往往比内层少一条乘法更值得先做。风险在于病态 conic、根号的小负舍入、切线相交、像素中心约定和整数端点。浮点实现应保守，不允许为了少发一个 tile 而丢掉任何参考实现会接受的像素；对非正定或溢出情况应有明确回退。

本分支与本任务。 本 fork 的 `isect_tiles.cu` 已有 opacity-aware compact-box 和解析 sweep，`compact_box_impl="sweep"` 是实现选择的默认值，但是否启用 compact-box 仍是另一个开关。产品封装原来已经默认启用。相机 0 的消融显示相交对约从 16.05M 降为 9.52M。这是已有能力的验证，不是本任务新增的收益。

2.3.3 第三类：8×4 子块掩码和 48 字节结构体

一个 16×16 tile 有 256 个像素。让 8 个 warp 分别拥有一个 8×4 矩形，可把每个高斯对 8 个子块的几何重叠关系压成一个 8 位掩码。对于 warp w，掩码的第 w 位为 0，意味着整个 warp 可以完全跳过该高斯。这比 32 个线程先分别计算高斯再投票更早地消除了工作。

把 32 个 staging 高斯看作一个小批次：lane ℓ 检查第 ℓ 个高斯是否覆盖当前 warp，ballot 把 32 个布尔值组成工作表，再用最低置位迭代。

```
unsigned work = __ballot_sync(0xffffffffu,
    lane < batch_count && ((batch_mask[lane] >> warp_id) & 1));
while (work != 0) {
    int j = __ffs(work) - 1;
    work &= work - 1;  // preserve increasing splat/depth order
    blend_shared_splat(j);
}

struct alignas(16) Splat48 {
    float4 center_and_aux;
    float4 conic_and_opacity;
```

```
    float4 rgb_and_aux;
};  // Illustrative field assignment: 3 * 16 bytes.
```

48 字节结构体将二维中心、conic、颜色和不透明度连续存储，使单线程可以用 3 次 16 字节宽读取完成 staging。此处的收益是减少散乱的标量地址与加载指令，不是宣称每个 warp 只产生 48 字节流量。相邻线程访问不同高斯时，实际事务仍取决于布局、缓存扇区和访问模式；`float4` 名称本身也不能保证最终机器码一定向量化。

瓶颈与风险。 掩码针对 warp 级空转，打包针对 staging 指令和局部性。计算 mask 本身也有成本，必须由跳过的工作偿还。布局改变还涉及 packed camera、多通道特征、对齐和 ABI。几何 mask 只是“可能覆盖”，不是 “确实贡献”，不能拿来直接证明该高斯的梯度非零。

本分支与本任务。 本任务没有移植文章的精确 conic 子块 mask，也没有交付 48 字节打包结构。曾测试 8×4 像素所有权加保守 AABB mask 的前向原型，FP32 版本约有 $1.020\times$ 的 raster 收益，但小且有噪声，并增加专用循环和回退，最终撤回。这不是对文章精确算法的反证：AABB 更松，病态 conic 回退更多，原型并不等价。

2.3.4 第四类：两阶段排序，把重复的深度工作提到前面

传统方法为每个相交对构造 $(\text{tile}, \text{depth})$ 宽键，在 P 个元素上排序。同一个高斯的深度会随其覆盖的 tile 数重复参与排序。若先把 M 个可见高斯按 32 位深度排序，再按此顺序发射 tile 对，整个相交数组已经具有全局深度顺序。最后对 tile ID 做**稳定**排序，就能在按 tile 分组的同时保留组内深度顺序。

$$\text{stableSort}_{\text{tile}}\left(\text{emit}\left(\text{sort}_{\text{depth}}(\text{visible Gaussians})\right)\right) \implies \text{每个 tile 内深度有序.} \tag{2.13}$$

```
visible = filter(count_tiles > 0);
visible = stable_sort_by_depth(visible);
offsets = exclusive_scan(count_tiles[visible]);
emit_at_scanned_offsets(visible, offsets, tile_keys, gaussian_ids);
stable_radix_sort(tile_keys, gaussian_ids);  // narrow tile key
find_tile_ranges();
```

扫描偏移很重要：如果每个高斯用无序的全局原子 append 发射，“深度顺序发射” 就不再成立。相等深度也需要沿用基线 tie-break；透明混合并不交换，稳定性是图像语义的一部分。正的有限相机深度通常可以用相应浮点位序构造单调键；负深度、NaN 或不同编码不能靠强制类型转换草率处理。

瓶颈、风险和状态。 文章用 32 位可见深度排序加 16 位 tile 稳定排序减少 radix pass 和字节流量。收益依赖 P/M、可见过滤成本和排序库。16 位 tile 键要求整个命名空间可表示，多相机 tile 编码也必须算进去。本任务相机图像仅有 3,600 个 16×16 tile，但通用 API 不能因此永久假定 16 位足够。本 fork 仍走原有排序路径；本任务只做源码评估，没有实现或测量该步骤，不能报告其增量收益。

2.3.5 第五类：少做 SH、宽加载、base-2 指数与标量反向状态

可见性之后才读 SH

三阶 SH 有 16 个基函数，每个 RGB 三通道，一共 48 个 FP32，即每高斯 192 字节。若先算完整 SH 再发现高斯在视锥外，就浪费了带宽和寄存器。先做便宜的可见性判断，再读取 SH，是按数据依赖重排工作，不是数值近似。连续系数用宽加载还能减少加载指令，但基函数与 RGB 的交错布局需要正确处理。

```
Projection p = project_geometry(id);
if (!visible_and_has_tiles(p)) return;
float4 v0 = load_aligned_SH_group(id, 0);
float4 v1 = load_aligned_SH_group(id, 1);
evaluate_visible_color(v0, v1, /* more groups */);
```

把自然指数改写为硬件 base-2 指数

令 $\beta = -\frac{1}{2}\log_2 e$，则

$$\exp(-q/2) = 2^{\beta q}. \tag{2.14}$$

把 conic 的缩放提前到预处理，可以让内层直接形成 base-2 指数输入，使用硬件快速指数路径。要注意交叉项 $2bd_xd_y$ 的系数约定：有的布局已把 2 合进去，有的没有，拷贝三个数字而忽略该差异会渲染出错误椭圆。

瓶颈与风险。 可见性门控减少 M 对应的 SH 流量，宽读减少 LSU 发射压力，指数重排减少内层转换工作。但是快速指数有近似和 FTZ 语义，系数预缩放也会改变舍入。若原有 __expf 加 fast-math 已经生成同样的硬件路径，仅把函数名改为 exp2 未必会少任何指令，不能从源代码名称推断收益。

把反向的 RGB 状态投影成一个标量

设像素的上游 RGB 梯度为 $\boldsymbol{u} = \partial L/\partial \boldsymbol{I}$，当前高斯之后、以该处透射率归一的尾部颜色为 $\boldsymbol{B}_i$，其中包含背景。在固定当前像素的 $\boldsymbol{u}$ 下，

$$\frac{\partial L}{\partial \alpha_i} = T_{i-1}\left(\boldsymbol{u}\cdot\boldsymbol{c}_i - \boldsymbol{u}\cdot\boldsymbol{B}_i\right). \tag{2.15}$$

后续计算只需要尾部颜色在 $\boldsymbol{u}$ 方向上的投影。因此令 $b_i = \boldsymbol{u}\cdot\boldsymbol{B}_i$，用一个标量递推取代 RGB 三个分量：

$$b_{i-1} = \alpha_i(\boldsymbol{u}\cdot\boldsymbol{c}_i) + (1-\alpha_i)b_i. \tag{2.16}$$

这没有删掉颜色梯度本身：$\partial L/\partial \boldsymbol{c}_i = T_{i-1}\alpha_i\boldsymbol{u}$ 仍然是向量。标量化的是反向遍历的运行状态。上游梯度在像素内部固定、尾部递推线性，才允许这种投影交换。

```
float behind_dot = dot(upstream_rgb, background_rgb);
for (splat : reverse_list) {
    float color_dot = dot(upstream_rgb, splat.rgb);
    float d_alpha = T_before * (color_dot - behind_dot);
    behind_dot = alpha * color_dot + (1 - alpha) * behind_dot;
    accumulate_color_and_geometry_gradients(d_alpha);
}
```

本分支与本任务。 本 fork 的 SH VJP 已按颜色通道分工，不能把文章的 SH 寄存器模型原样套用。本任务未新增交付 forward SH 宽加载或 base-2 重排；现有快速指数路径也不算新收益。标量颜色投影及背景点积外提确实实现并测试过，112 个小例通过，但对应一轮 raster 中位数从 13.945 ms 变为 14.496 ms，没有收益，最终撤回。代数上减少状态并不保证在特定编译器和 GPU 上更快；依赖链和寄存器分配同样重要。

2.4 s02：让归约形状匹配梯度宽度

2.4.1 halving reduce-scatter 的核心不变量

butterfly 计算了所有 lane 都不需要的重复结果。我们真正需要的是：每个分量在某个 lane 上有完整总和，然后由那个 lane 写回。不需要 32 个 lane 都持有全部 9 项。这就是 reduce-scatter：在归约像素维度的同时，把输出分量分散。

用 4 个线程、4 个分量的教学模型说明。第一步让线程 $(0,2)$、$(1,3)$ 配对，每对中的一个保留分量 0、1 的局部和，另一个保留 2、3。第二步在拥有同一分量区间的线程间配对，继续把区间一分为二。最后每个线程只保留一个完整分量的和。

$$\underbrace{4\times 4}_{\text{每线程持有所有分量}} \longrightarrow \underbrace{4\times 2}_{\text{合并两线程的部分和}} \longrightarrow \underbrace{4\times 1}_{\text{每分量唯一拥有者}}. \tag{2.17}$$

9 项不是 2 的幂，可以按实现的所有权规则补零、切分奇数区间，并保证虚拟项不写回。文章给出的有效窗口长度为

$$9 \longrightarrow 5 \longrightarrow 3 \longrightarrow 2 \longrightarrow 1 \longrightarrow 1, \qquad 5+3+2+1+1=12. \tag{2.18}$$

因此从每 splat 45 条 shuffle 降至约 12 条，最后由 9 个拥有者在同一 warp 原子指令中写出 9 项。全局原子地址更新的总项数仍为 9，并没有神奇地合并成一个地址。

```
// Conceptual collective, not a literal shuffle implementation.
Window w = initial_gradient_window(local_grad);
for (int stride = 16; stride != 0; stride >>= 1) {
    Ownership next = split_components_between_partners(w, lane, stride);
    Window remote = exchange_partner_selected_half(w, next, stride);
    w = add_selected_local_and_remote(w, remote, next);
}
if (owns_real_component(lane))
    atomicAdd(component_address(lane), owned_sum(w));
```

写真正的 shuffle 代码时，源 lane 要发送对方所需的分量，不是所有 lane 读取同一个数组下标就会自动正确。最有效的验证是给每个输入 (ℓ, j) 唯一标记，在小模型中追踪每一步的分量所有权，证明每个 G_j 覆盖恰好 32 个输入且只写一次。

2.4.2 两 splat half-warp folding：填满尾部的空闲位置

9 个输出连半个 warp 都放得下。单 splat 的后期归约仍要跨 32 lane 合并，却已不能再把一项拆成半项。文章将两个连续 splat A、B 的像素贡献同时保留：先跨上下半 warp 配对，把 A 的 32 像素贡献折叠进低半 warp，把 B 折叠进高半 warp；再在两个 16-lane 组内并行执行 reduce-scatter。

$$A'_{\ell j} = A_{\ell j} + A_{\ell+16,j}, \quad 0 \le \ell < 16, \tag{2.19}$$

$$B'_{\ell j} = B_{\ell j} + B_{\ell-16,j}, \quad 16 \le \ell < 32. \tag{2.20}$$

两组各自完成 16 像素局部和的再归约后，就得到各自完整的 32 像素结果。初次 folding 交换 9 项，后面的半 warp 窗口归约花费 $5+3+2+1=11$，合计每对 20 次，即平均每 splat 10 次，而不是单独执行两次 12。

预期瓶颈与风险。 收益来自更少 shuffle 和更平行的写回，但代价是同时存活的两个 splat 状态。寄存器数过高会降低驻留率，甚至 spill 到 local memory。文章的 SM80 启动约束是其编译结果的一部分，不是跨 GPU 的最佳常数。还要处理奇数结尾、两 splat 不同的有效像素集合、absgrad 和非 RGB 通道。

本分支与本任务。 最终只保留**单 splat halving reduce-scatter**，两 splat folding 未移植。窄通道路径支持至多 4 个颜色/特征通道，并把可选绝对中心梯度纳入实际归约窗口；通道更宽时沿用原实现。因此文章的 “9 项、12 次” 是教学例，不是本 fork 所有配置的指令计数。

一次后续检查发现 4×4 tile 只有 16 个线程：假定完整 warp 的拥有者可能根本不存在，导致本应非零的不透明度梯度变零。**传入 active mask 不能创造缺失的拥有者。**最终提交 88513ea 对 `block_size % 32 != 0` 的整个 block 回退到旧路径，覆盖 4×4 及 6×6 等配置。最终 RGB backward 报告 48 个寄存器、无 stack/local memory；早期 scatter-only 的静态指令计数不能冒充含回退分支的最终二进制总计数。

2.5 s03：SH 寄存器流式化与协作清零

2.5.1 为什么 SH 不需要同时保存全部输入和输出

设方向为 $\boldsymbol{d}$，SH 基函数为 Y_k，RGB 系数为 $\boldsymbol{s}_k$：

$$\boldsymbol{c}(\boldsymbol{d}) = \sum_k Y_k(\boldsymbol{d}) \boldsymbol{s}_k, \tag{2.21}$$

$$\frac{\partial L}{\partial \boldsymbol{s}_k} = Y_k(\boldsymbol{d}) \boldsymbol{u}, \tag{2.22}$$

$$\frac{\partial L}{\partial \boldsymbol{d}} = \sum_k (\boldsymbol{u} \cdot \boldsymbol{s}_k) \nabla_{\boldsymbol{d}} Y_k(\boldsymbol{d}). \tag{2.23}$$

系数梯度只依赖该基函数和上游梯度，写完就可以释放寄存器；方向梯度只需要一个三分量运行和。于是没有必要同时保留 48 个输入系数和 48 个输出梯度。颜色非负截断等前向操作的导数掩码应先作用于 $\boldsymbol{u}$，不能在流式化时丢掉。

文章把 16 个三阶基函数分成 4 组，每组 4 个，即 12 个 RGB 浮点数。12 同时能被 RGB 宽度 3 和向量读取宽度 4 整除，适合作为一个寄存器窗口。

```
float3 d_direction = make_float3(0, 0, 0);
for (int group = 0; group < 4; ++group) {
    float v[12];
    load_three_float4(group, v);
    for (int k = 0; k < 4; ++k) {
        Basis y = basis_and_direction_derivative(4 * group + k);
        store_SH_gradient(4 * group + k, y.value * upstream_rgb);
        d_direction += dot(upstream_rgb, rgb(v, k)) * y.derivative;
    }
}
propagate_normalized_direction_vjp(d_direction);
```

预期瓶颈与风险。 这针对活跃寄存器数量和由此产生的驻留率，而非减少 SH 的数学项数。文章的 FP32 backward 寄存器数从 152 降至 104，是 A100 具体内核的结果。窗口太小会增加加载与索引，太大又失去驻留率收益。方向归一化的雅可比仍需正确回传，不能把方向导数直接当作位置导数。

2.5.2 移除 bulk memset 不是允许输出未初始化

若完整参数梯度缓冲区先清零，再由可见高斯覆盖，每个活跃元素被写了两次。文章让活跃项直接写最终梯度，非活跃项由内核写零，移除单独的全量 memset。但有一个所有权前提：这些输出必须是最终赋值，而不是依赖预先清零的原子累加目标。光栅阶段共享的累加梯度缓冲区不能不加区分地照搬该方法。

朴素地让每个线程清自己高斯的 192 字节 SH 行，会使 warp 同一条存储指令的地址相隔 192 字节。A100 的缓存按 32 字节 sector 传输，许多 sector 只写一小部分。更好的方式是让 warp 协作线性扫描 32 个高斯的连续 SH 区域，以 16 字节块为单位：

```
unsigned active = __ballot_sync(0xffffffffu, gaussian_is_active);
// 32 rows, each with 12 float4 chunks for degree-3 RGB SH.
for (int chunk = lane; chunk < 32 * 12; chunk += 32) {
    int owner = chunk / 12;
    if (((active >> owner) & 1) == 0)
        store_float4_zero(warp_SH_base + chunk);
}
// Active rows are fully assigned by their VJP owners.
```

相邻 lane 总访问相邻 16 字节块，因此合并性改善，而非活跃行仍有正确的零梯度。尾 warp、实际系数数 K 大于当前阶数所用数量、未启用方向梯度等情况，都需要定义哪些元素由 VJP 写、哪些由零填充写，确保没有遗漏和重叠竞态。

本分支与本任务。 本 fork 的 SH backward 是 $N \times 3$ 个线程，每线程处理一个颜色通道，在 VJP 中直接写系数梯度，不存在文章同样的 “48 输入加 48 输出都常驻” 结构。数据又是一阶 SH，不能套用三阶的带宽与寄存器收益。本任务实现过选择性协作清零，288 个 SH 回归例覆盖阶数 0–4、mask、超额 K 和方向梯度开关，全部通过；但配对完整渲染测量没有可靠收益，一轮中位数为 64.056 ms 对 64.973 ms，最终撤回。`zeros_like` 改成 `empty_like` 而不负责所有未写元素，绝不是这一优化的正确简化。

2.6 s04：前向知道谁贡献过，就不要让反向重新猜

2.6.1 几何重叠与真实贡献的区别

一个高斯可以与 tile 椭圆相交，却因为低权重或遮挡没有对任何像素产生实际混合。几何 mask 是保守候选集合；贡献 mask 是执行了基线接受逻辑之后得到的事实。记 tile 中第 i 个高斯的 warp 贡献掩码为

$$m_i = \sum_{w=0}^{7} 2^w \mathbf{1}\{\text{warp } w \text{ 至少一个像素实际混合了 } i\}. \tag{2.24}$$

当 $m_i = 0$，该高斯在这个 tile 内可以从反向列表中删去。注意这里要求的是 “实际混合”，不是仅满足 $\alpha \geq \tau$；已经终止的像素不能使它成为贡献者。

2.6.2 原位压缩和停止索引的重映射

设原列表中的贡献标记为 $v_i = \mathbf{1}\{m_i \neq 0\}$，稳定压缩后的位置为

$$r_i = \sum_{j<i} v_j. \tag{2.25}$$

保留原顺序，把第 i 项移到 r_i，于是压缩后的顺序仍是深度顺序。一个像素原来的停止位置也必须用相同前缀计数映射；否则反向可能在错误的 splat 停止，甚至使用一个压缩前位置索引压缩后数组。

```
stage_whole_batch_before_overwriting_source();
for (splat : batch) {
    unsigned cmask = collect_actual_blend_bits_from_all_warps(splat);
    keep[splat] = (cmask != 0);
}
rank = exclusive_scan(keep);
```

```
for (splat : batch) {
    if (keep[splat])
        list[tile_begin + prior_kept + rank[splat]] = pack(splat);
}
remap_per_pixel_last_indices_using_same_prefix();
```

“原位” 的安全依据是输出写指针不超过当前读指针，并且当前批次已完整 staging。若某 warp 提前覆盖另一个 warp 尚未读取的元素，数学上的稳定压缩依然会在实现中变成数据竞争。tile 的切片、跨 batch 的计数和同步点都必须纳入证明。

2.6.3 高字节 cmask 与全局 touched

文章将 8 位贡献掩码打包进 32 位列表项的高字节：

$$\text{packed} = (m_i \ll 24) \mid \text{id}, \qquad \text{id} = \text{packed} \,\&\, (2^{24} - 1). \tag{2.26}$$

反向直接使用该掩码，避免重新计算子块几何边界。另设每个高斯的 splat_touched 标记，只有实际贡献过的高斯才运行昂贵的预处理 VJP。这会把 “视锥可见但完全遮挡” 的高斯从 SH backward 工作集中移除。

24 位限制不能靠场景平均规模回避。 这种格式只容纳 0 至 $2^{24} - 1$ 的 ID。本任务有 20,789,814 个高斯，超过 $2^{24} = 16,777,216$；最终实测列表 ID 达到 20,789,812。直接截低 24 位会把某些高斯别名到另一些高斯，输出可能仍然看似合理，却产生错误梯度。安全设计可以采用独立 mask 缓冲区、64 位条目、可见集合重映射，或明确的按规模回退；每一种都有额外字节、扫描和接口代价。

预期瓶颈与风险。 前向额外做投票、计数和写回，以换取反向少 staging、少求值、少 VJP。因此纯前向可能更慢。touched 还需要正确初始化、跨 tile 更新与跨内核可见性；不能仅因为多个线程都写 1 就忽略内存模型或竞争检查。多通道分块调用、多个消费者复用原列表时，修改输入列表尤其危险。

2.6.4 本任务选择更小的改动：只裁掉完全无用的尾部

完整原位压缩、打包 cmask 和新的 touched 契约都没有移植。本任务利用已有逐像素 last_ids 求 tile 内最大停止索引，只删去所有像素都未使用的后缀：

$$L_{\text{tile}} = \max_{p \in \text{tile}} L_p. \tag{2.27}$$

反向从这个边界开始，不再 staging 更深的完全遮挡后缀；中间没有贡献的洞仍保留。该方案不修改共享列表，不压缩 ID，也不改变停止索引的坐标系，适合本 fork 的通道分块复用。它不是文章完整压缩的同义词。

空 tile 必须先统一返回；从包含端点变成排他端点的加一应先提升整数宽度，避免靠近整数上界时溢出。初期隔离实验中，在 scatter 之上加入后缀裁剪有明显收益，因而最终保留；但最终总倍率来自重建后二进制的重新配对测量，不能把早期倍率与后期倍率相乘来宣称最终性能。

2.7 s05：分歧、谓词化与精确的数值分支边界

2.7.1 为什么 “少执行线程” 未必意味着少发射指令

warp 中各 lane 共享指令发射。若 31 个像素都 miss、1 个像素 hit，普通分支下 warp 仍要发射混合和梯度指令，只是其中多数 lane 被禁用。若候选 mask 已经保证至少一个像素有用，逐像素的多个

continue 常常只增加重汇合与谓词管理。让 miss lane 的有效 $\alpha = 0$，可以把递推写成连续直线代码：

$$\alpha_{\text{eff}} = \begin{cases} \alpha, & \text{hit}, \\ 0, & \text{miss}, \end{cases} \qquad \Delta \boldsymbol{C} = T\alpha_{\text{eff}}\boldsymbol{c}, \qquad T' = T(1-\alpha_{\text{eff}}). \tag{2.28}$$

在有限数、没有副作用的理想代数中，miss lane 的状态完全不变。

```
float power = exact_shared_quadratic(conic, dx, dy);
float raw = opacity * fast_exp(power);
float capped = min(reference_cap, raw);
bool hit = active && reference_power_test(power)
                  && reference_cutoff_test(capped);
float alpha = hit ? capped : 0.0f;
// Reference termination ordering must still be applied here.
C += (T * alpha) * color;
T *= (1.0f - alpha);
```

本 fork 没有文章同样的先决条件。 本任务未交付精确 warp 贡献 mask，所以候选高斯仍可能整 warp 都 miss。保留 warp-wide all-miss 跳过才合理；不能借文章的结构不变量，删除本实现中仍有价值的统一分支。宽通道路径也保留原来的条件求值，避免所有 lane 对大量特征执行无效运算。

2.7.2 精确舍入：前后向必须在同一侧跨过 cutoff

若前向把某像素判为 hit，而反向因 FMA 融合方式不同判为 miss，反向会漏掉一次 $T/(1-\alpha)$ 恢复，后面整条反向递推都会受影响。问题不只是一个高斯边缘上极小的局部误差，而是离散控制流变化。

因此文章用显式 round-to-nearest-even 操作固定二次型求值的表达式树。下面只是展示固定树的方法；实际树必须与目标基线一致：

```
// Illustrative RN tree; use exactly the same helper in fwd and bwd.
float xx = __fmul_rn(dx, dx);
float xy = __fmul_rn(dx, dy);
float yy = __fmul_rn(dy, dy);
float axx = __fmul_rn(a, xx);
float bxy = __fmul_rn(two_b, xy);
float cyy = __fmul_rn(c, yy);
float q = __fadd_rn(__fadd_rn(axx, bxy), cyy);
float power = __fmul_rn(-0.5f, q);
```

不能把这个教学树直接覆盖到已有 fork：若原基线使用另一种乘加树，固定为新树虽然让新前后向一致，仍可能改变与旧实现的 cutoff 归属。本任务没有交付新的前向表达式、conic 预缩放或 base-2 重排，而是保持现有前向语义进行差分验证。所谓 “exact rounding” 是操作舍入可控，不是高斯函数的实数精确求值。

2.7.3 clamp、cutoff 和反向零贡献不是同一件事

cutoff 是 “不接受这个混合项”；cap 是 “接受，但把 alpha 饱和”。对于 cap 已饱和的像素，颜色梯度仍然存在，而通过 alpha 到几何、不透明度的导数应按参考实现的饱和导数规则门控。不能把这两类掩码合成一个随意的 hit。

另外 IEEE 浮点满足 $0 \times \infty = \text{NaN}$，并不满足 “乘了零就是零”。有限 conic 也可能在中间乘法溢出；被拒绝 lane 的上游梯度还可能是非有限值。若先无条件算出 NaN，再乘零，归约就会污染其他像素。

```
Grad g = exact_zero_grad();
if (hit) {
```

```
    update_backward_color_state();
    g.color = accepted_color_vjp();
    if (reference_alpha_derivative_is_enabled(raw)) {
        g.geometry = geometry_vjp();
        g.opacity = opacity_vjp();
        g.abs_mean = optional_abs_mean_vjp();
    }
}
reduce_gradients(g);  // invalid lanes really contribute zero
```

这说明安全实现可以只让某些递推谓词化，仍对危险导数求值保留显式门控。追求源代码 “没有 if” 不是目标；减少真实机器码分歧并保留语义才是目标。NaN 的比较、最小值指令行为、clamp 边界的等号约定也要继承基线。

本任务结果。 窄通道 backward predication 在加固后保留，前向完全不变。加固内容包括拒绝 lane 不更新颜色运行缓冲，拒绝或饱和 lane 不计算几何、不透明度及 absgrad 导数。一次最终集成消融中，相机 0 在 scatter 加 trim 之上约有 $1.038\times$ 的增量 raster 收益；这是该测量序列的结果，不是任意场景或最终全部相机的独立倍率。有限但溢出的 conic、非有限拒绝 lane 和非默认流已进入最终回归矩阵。

2.8 s06：多行映射与执行流水线平衡

2.8.1 每线程两个竖直像素：先在线程内合并

文章将一个 warp 的像素覆盖从 8×4 改为 8×8，每个线程负责同一列的两个相邻像素。一个 16×16 tile 从 256 线程变为 128 线程，即由 8 个 warp 变为 4 个 warp。两个像素共享 d_x、横向二次项和高斯属性设置。反向先在线程寄存器中把两个像素的梯度相加，再做 warp 归约，故每 tile 的归约次数和对应原子小计数减半。

```
Pixel top = pair_top_pixel(lane, warp_id);
Pixel bottom = top + vertical_offset(1);
Column col = setup_shared_x_terms(splat, top.x);
State s0 = update_one_row(col, top, state0);
State s1 = update_one_row(col, bottom, state1);
Grad local = s0.grad + s1.grad;
warp_reduce_scatter(local);
```

两个像素仍有独立颜色、透射率、上游梯度和终止状态，不能因为共享列就共用一份像素状态。屏幕底边和不整齐图像边缘可能只有其中一个像素有效。

2.8.2 warp 私有 staging：去掉等待最慢邻居的时间

旧结构的所有 warp 共用一个 tile 级共享 batch；即使某 warp 的区域很空，也必须在 `__syncthreads()` 等待最繁忙的 warp。文章在 backward 给每个 warp 分配独立的 32-splat staging 切片，只在本 warp 内同步，并根据贡献 mask 独立遍历。这不是把同步函数名直接改成 `__syncwarp()`：共享内存的所有权和生产消费范围必须一起重设计，否则其他 warp 可能覆盖仍在读取的数据。

流水线视角。 文章用合成场景的 Nsight Compute 指标说明，某些步骤受 block barrier 拖累，另一些步骤受 FMA、ALU 或总发射率限制。各执行管线都不到满载，并不表示没有瓶颈：如果 scheduler 每周期只能发射有限条指令，混合整数、浮点、指数和加载指令的总量可以先耗尽发射带宽。要区分 “少 FMA” 和 “少总指令”，也要区分运行时动态计数与静态 SASS 中出现了多少条指令。

2.8.3 把不透明度并入指数，复用列常数

对 $o > 0$，有

$$o\,2^{p} = 2^{p+\log_2 o}. \tag{2.29}$$

预处理保存 $\log_2 o$，列设置时将它合进横向常数，每行指数输出就直接是 α_{raw}，少一次逐行不透明度乘法。反向几何使用的 log-alpha 导数也可保留 o 因子，到输出不透明度梯度的拥有者处再恢复相应比例。

代数相同不等于浮点位相同：$o = 0$、极小 o、FTZ、先乘后 cap 与先加 log 再指数，都会涉及边界。文章还结合列范围检查减少逐像素 power 判断，这依赖其完整设计，不是只替换一个指数就能继承的性质。

2.8.4 竖直两行矩：把乘法延后到归约之后

取文章的符号约定，两个像素的相对纵坐标为 y 与 $y-1$，相应 log-alpha 梯度为 g_0, g_1。定义三个矩：

$$m_0 = g_0 + g_1, \tag{2.30}$$

$$S_y = g_0 y + g_1(y-1) = m_0 y - g_1, \tag{2.31}$$

$$S_{yy} = g_0 y^2 + g_1(y-1)^2 = m_0 y^2 - g_1(2y-1). \tag{2.32}$$

这些量足以构造多种均值和 conic 梯度。如果竖直 shuffle 连接的是同一像素列，则各线程有同一个 d_x，可以利用

$$\sum_{\ell} d_x m_{0,\ell} = d_x \sum_{\ell} m_{0,\ell} \tag{2.33}$$

把横向乘法延后到竖直部分归约之后。这里节约来自几何映射提供的公共因子。若修改 lane 编号使配对不再同列，上式的公共 d_x 假设就失效了。

2.8.5 终止状态编码与 BF16 的不同性质

文章还用透射率的符号位编码像素是否已终止，避免单独维护某些布尔状态：负号表示完成，绝对值仍保存输出所需透射率。这只适用于 T 原本非负的约定；所有使用点都得区分 “存储状态” 和 “物理透射率”，背景混合或恢复导数时不能误用负值。负零、下溢和最终取绝对值也需要明确语义。

BF16 SH 则属于另一类优化：通过降低系数存储宽度减少带宽，而不是单纯重排计算。BF16 保留与 FP32 相同数量的指数位，但有效精度更低；应说明是否以 FP32 累加、梯度如何保存、优化器是否有主副本，以及参数更新时何处发生量化。文章在其测试中报告相同到 0.01 dB 的 PSNR，不等于任意场景的逐元素结果相同，更不能自动证明任意优化器的训练轨迹不变。

预期瓶颈、风险和本任务状态。 多行减少设置、归约和原子次数，warp 私有 staging 减少 barrier 等待；代价是每线程更多像素状态和更复杂 mask 映射。RGB+ED 或宽特征增加寄存器压力，可能使 RGB 上的收益反转。BF16 对三阶 SH 更有动力，本任务一阶系数流量本来就小。上述多行、私有 staging、管线数学重排、符号编码和 BF16 都只做了源码/硬件适用性评估，**未实现、未测量、未作为交付内容**。不能给这些步骤填入来自 A100 的预期倍率，冒充 4090 上已经获得的收益。

2.9 负结果：失败实验教会了什么

文章单列了 17 个负结果及硬件陷阱。它们的教学价值不是形成一份永远禁止使用的技巧名单，而是说明同一个局部优化可以撞到另一个约束。下面按因果关系整理这些陷阱，并与本任务实际撤回的实验区分；不把未在本任务运行的方案说成已经失败。

2.9.1 驻留率不是越高越好，寄存器也不是越少越好

强制压低寄存器可以增加理论驻留 warp 数，却可能导致 spill，产生 local memory 读写；反过来，放宽寄存器预算可以消除 spill，却可能跨过驻留 block 的台阶。正确的问题不是 “occupancy 达到多少”，而是新增的 warp 是否能隐藏当前瓶颈的延迟，以及为它付出了多少额外流量。一个依赖链很长的指数–除法递推，也不会仅靠少两条独立加法就完全变快。

因此应该联合记录寄存器、spill、活跃 warp、内核时间，不能只展示源代码行数减少。本任务最终保留的 reduce-scatter 需要更多局部部分和，但仍无 local memory；两 splat folding 则没有测量，不能假设进一步减少 shuffle 必然更快。

2.9.2 共享内存缓存并不免费，更多并行也可能引入等待

把重复读取放进共享内存可能节约全局加载，也增加 staging、屏障、bank conflict 和容量占用。将工作拆成更多 warp 可能改善覆盖，却增加归约次数；把工作合并给更少线程可能降低设置成本，却增加寄存器和长尾。文章的方形多行映射与私有 staging 是配套变化，不是两个可以无条件相加的独立倍率。

```
// A useful experiment record contains both intent and a rejection gate.
Hypothesis h = "less staging or fewer issued instructions";
Candidate c = change_one_mechanism(h);
if (!differential_correctness(c)) reject(c);
else if (!paired_repeated_target_timing_improves(c)) reject(c);
else retain_with_documented_scope(c);
```

2.9.3 缓存局部性和排序的收益会互相影响

宽键变窄会少搬字节，但新增过滤、扫描和中间数组也有成本；tile 更精确会减少排序规模，但也可能让其他固定开销占比升高。重新排列高斯以改善某阶段的连续读，可能让另一个阶段的梯度写回更分散。L2 命中率单独变高并不是充分证据：如果总请求数和指令数一起增长，绝对时间仍可能更差。

文章的两阶段排序针对重复深度排序这一具体问题。本任务没有实现它，所以本任务的 “未交付” 不能被写成 “实测无效”。同样，AABB 子块原型的弱收益也不能用于否定文章精确 conic mask 加紧凑布局的组合。

2.9.4 本任务三个明确撤回的实验

1. **标量颜色累加。**数学线性化成立，112 个小回归通过，但对应 raster 测量没有收益，故恢复原计算。这里的结论是目标负载上不值得交付，不是标量投影公式错误。
2. **SH 选择性协作清零。**288 个专门测试通过，但完整渲染配对结果没有可靠增益。索引、除法、掩码开销和非活跃行比例可能抵消减少的写流量；这是解释假说，而不是已经由硬件计数器证明的瓶颈诊断。
3. **前向 AABB warp culling。**通过小回归但收益小、噪声显著，专用循环和病态边界回退增加维护负担。早期 FP64 边界原型还暴露了与浮点溢出基线兼容的风险，所以没有交付任何前向源码修改。

此外，部分 warp 缺失拥有者不是一次可以接受的性能负结果，而是必须修复的正确性缺陷。最终引入回退并重新构建、重新测量，先前二进制退出交付，不能继续使用它更漂亮的历史数字。判定性能之前必须先确认测量的就是待交付二进制。

2.10 把文章读完之后：怎样评价这次移植

2.10.1 交付范围与最终测量

最终提交 88513ea 的内核变化仅在 backward rasterization 和归约辅助代码：窄通道 reduce-scatter、遮挡后缀裁剪、安全的 alpha predication，以及部分 warp 回退。前向和 SH 源码已恢复，完整压缩、两阶段排序、two-splat folding、多行和 BF16 均未移植。该状态是隔离的优化/审查分支，不是已经替换用户安装包或合并到主工作区的生产发布。

最终重建后的全量场景 RGB 配对结果如下。raster 指前后向光栅 API；完整渲染还包括投影、SH、分箱和相应反向等。数字是作业中位数的中位数，不是取最快一次。

相机索引	raster 倍率	完整可微渲染倍率
0	$1.337\times$	$1.100\times$
2552	$1.648\times$	$1.016\times$
5103	$1.363\times$	$1.039\times$

这只有三个视图，不是全部 5,104 个相机。相机 2552 的两个完整渲染配对方向不一致，其小幅汇总收益仍不确定。产品 RGB+ED 加 absgrad 设置在相机 0 上为 $1.311\times$ raster、$1.081\times$ 完整渲染；不能用 RGB 三通道结果代替四通道产品设置。未测试优化器、损失、训练循环整体收益或训练收敛。

2.10.2 正确性证据也需要分层

最终二进制在默认流和非默认流上各通过 136 个差分测试，memcheck 和 synccheck 均报告 0 错误。全量场景 RGB/alpha 与基线逐位相同，直接屏幕空间梯度的最差 relative-L2 为 2.62×10^{-6}（RGB）及 1.48×10^{-6}（RGB+ED），低于保持不变的 10^{-4} 门槛。

但是完整参数路径的单次 raw-quaternion 梯度比较约为 1.63×10^{-4}，**没有通过严格的** 10^{-4} **门槛**。重复原子累加噪声和近退化投影会放大误差，是理解现象的上下文，而不是豁免阈值的理由。固定 16 次运行、CPU FP64 累积的平均梯度检查通过，四元数平均 relative-L2 分别为 4.57×10^{-5} 和 3.50×10^{-5}；这只支持均值一致性，不把单次失败变成通过，更不构成收敛证明。

2.10.3 一套可以带走的思考顺序

1. **固定语义。**先写清参数激活、像素中心、cutoff、cap、终止顺序、相等深度、空输入与流语义。否则所谓加速可能只是换了问题。
2. **减少集合。**先问能否减少 P、实际遍历次数和不必要 SH 行，再问单次求值能否少一条指令。
3. **匹配硬件形状。**把 9 项输出映射到拥有者，把 SH 活跃窗口限制在必要范围，把共享数据与线程像素映射一起设计，而不是照抄某个函数名。
4. **检查代数背后的浮点。**线性投影、log 指数合并和零 alpha 都有有效范围；边界比较、非有限中间值及原子顺序仍然决定实现正确性。
5. **逐步过门。**每个改动独立建立假说、差分回归、消融和配对时间；拒绝无收益方案，保留不确定性，不把未实现部分写成已完成结果。

文章最值得迁移的不是 $7.30\times$ 这个数字，而是这条从算法工作量到硬件映射、从数值契约到证据边界的推理链。对本 fork 而言，较小且可验证的 backward 改动已经带来了可重复的 raster 收益；剩余方案应作为新的、独立的研究问题，而不是从 A100 图表中预支的性能承诺。

Chapter 3

从纸面公式到可运行的最小例子

3.1 动手算例：从一个像素到一次完整渲染

本章用可手算的数字和独立的小程序串起投影、合成、梯度归约与性能测量。所有时间均为教学用的假设值，不是新的 GPU 实测结果。Python 示例只依赖标准库，可以分别保存为一个 .py 文件运行；CUDA 部分会明确区分接口示例与算法伪代码。这里的四 lane 模型用于解释数据流，不表示 NVIDIA 的硬件 warp 只有四个线程。

3.1.1 算例一：从三维输入到 conic，再算两个 splat 的合成

先确认坐标与参数语义。 一个三维 Gaussian 的常见输入包括世界坐标均值 μ_w、尺度 s、旋转四元数 q、opacity o，以及颜色或 SH 系数。若保存的是 log-scale 和 opacity logit，须先做 $s = \exp(s_{\log})$ 与 $o = \mathrm{sigmoid}(o_{\log})$。四元数需要按约定归一化，且必须确认分量顺序；不能把 (w, x, y, z) 与 (x, y, z, w) 直接混用。

对列向量约定，相机的 camera-to-world 矩阵及其逆为

$$\mathrm{c2w} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}, \qquad \mathrm{viewmat} = \mathrm{c2w}^{-1} = \begin{bmatrix} R^T & -R^T t \\ 0 & 1 \end{bmatrix}.$$

这里假设 R 是正交旋转。例如相机位于世界坐标 $(1, 0, 0)$，朝向不变，则世界点 $(1, 0, 3)$ 变为相机点 $(0, 0, 3)$，而不是 $(2, 0, 3)$。实际调用还要检查相机轴方向、可见深度的符号与矩阵存储约定。

构造三维协方差并投影到二维的局部近似是

$$\Sigma_w = R(q)\,\mathrm{diag}(s_x^2, s_y^2, s_z^2) R(q)^T, \qquad \Sigma_{2D} = JW\Sigma_w W^T J^T + \epsilon I.$$

其中 W 是 view matrix 的旋转块，J 是在 Gaussian 中心处计算的透视投影雅可比；针孔模型为 $u = f_x x/z + c_x$、$v = f_y y/z + c_y$。所以相机变换、焦距、深度与低通正则项都会影响屏幕足迹。以下直接给出正定的二维逆协方差，即 conic：

$$\Lambda = \Sigma_{2D}^{-1} = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 1 \end{bmatrix}, \qquad \det \Lambda = 1.75 > 0.$$

一个像素的 alpha。 设投影中心是 $(10, 20)$，被采样的像素中心是 $(10.5, 19)$，因此 $d_x = 0.5$、$d_y = -1$。注意这里已经指定了采样位置，不要在代码中再次额外加半个像素。二次型为

$$\begin{aligned} Q &= a d_x^2 + 2b d_x d_y + c d_y^2 \\ &= 2(0.5)^2 + 2(0.5)(0.5)(-1) + 1(-1)^2 = 1, \\ \sigma &= Q/2 = 0.5, \\ \alpha_1 &= o_1 \exp(-\sigma) = 0.8 \exp(-0.5) \simeq 0.48522453. \end{aligned}$$

这里用 Q 表示二次型，以免与旋转四元数 q 混淆。该 alpha 高于 $1/255$，且未触及 0.999 上限，因此没有触发阈值或夹紧分支。交叉项中的系数是 $2b$，漏掉这个 2 会直接改变足迹和梯度。

前后两个 splat。 假设深度排序已经确定：前面的 splat 是红色 $c_1 = (1, 0, 0)$，后面的是蓝色 $c_2 = (0, 0, 1)$。第二个 splat 在本像素中心取样，$Q_2 = 0$、$o_2 = 0.25$，所以 $\alpha_2 = 0.25$。初始化 $T_0 = 1$、$C_0 = (0, 0, 0)$，则

$$
\begin{aligned}
w_1 &= T_0\alpha_1 = 0.48522453, & T_1 &= 0.51477547,\\
w_2 &= T_1\alpha_2 = 0.12869387, & T_2 &= 0.38608160,\\
C_2 &= (0.48522453, 0, 0.12869387).
\end{aligned}
$$

未加背景的累计颜色不是归一化的平均颜色。若背景为 $c_{\text{bg}} = (0.1, 0.1, 0.1)$，最终输出是

$$C = C_2 + T_2 c_{\text{bg}} \simeq (0.52383269, 0.03860816, 0.16730203).$$

三个权重满足 $w_1 + w_2 + T_2 = 1$。交换两个 splat 的深度顺序一般会改变颜色，所以排序并不是可以任意重排的实现细节。

```
from math import exp, isclose

dx, dy = 0.5, -1.0
a, b, c = 2.0, 0.5, 1.0
quad = a * dx * dx + 2.0 * b * dx * dy + c * dy * dy
alpha1 = min(0.999, 0.8 * exp(-0.5 * quad))
alpha2 = 0.25
T = 1.0
C = [0.0, 0.0, 0.0]
for alpha, color in [(alpha1, (1, 0, 0)), (alpha2, (0, 0, 1))]:
    weight = T * alpha
    C = [v + weight * x for v, x in zip(C, color)]
    T *= 1.0 - alpha
C = [v + T * 0.1 for v in C]
assert isclose(quad, 1.0)
assert isclose(alpha1 + (1.0 - alpha1) * alpha2 + T, 1.0)
print("alpha:", alpha1, alpha2)
print("RGB:", C, "remaining T:", T)
```

3.1.2 算例二：四 lane、四分量的归约与 reduce-scatter

反向光栅化时，同一个 Gaussian 会接收多个像素的梯度贡献。设四个 lane 分别持有四个分量，行对应 lane，列对应参数：

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 10 & 20 & 30 & 40 \\ 100 & 200 & 300 & 400 \\ 1000 & 2000 & 3000 & 4000 \end{bmatrix}.$$

我们需要的是列和 $(1111, 2222, 3333, 4444)$，不是每一行的和。普通 butterfly all-reduce 在每个 lane 上保留四个完整和，再选 lane 0 写出；reduce-scatter 则让最终的四个和分别属于四个 lane。

第一轮配对距离为 2。lane 0 与 2 配对，只保留分量 0、1；lane 1 与 3 同理。lane 2、3 分别保留分量 2、3。第一轮之后的局部数组为

$$
\begin{aligned}
&\text{lane } 0 \;:\; (101, 202) \quad \text{分量 } (0, 1),\\
&\text{lane } 1 \;:\; (1010, 2020) \quad \text{分量 } (0, 1),\\
&\text{lane } 2 \;:\; (303, 404) \quad \text{分量 } (2, 3),\\
&\text{lane } 3 \;:\; (3030, 4040) \quad \text{分量 } (2, 3).
\end{aligned}
$$

第二轮距离为 1，低位 lane 保留各自窗口的前半，高位 lane 保留后半。最终 ownership 是 lane 0 拥有分量 0 的 1111，lane 1 拥有分量 1 的 2222，lane 2 拥有分量 2 的 3333，lane 3 拥有分量 3 的 4444。

下面同时模拟两条路径。每轮先保存旧状态非常重要：shuffle 是同一轮的 lane 间交换，不能让 Python 的循环顺序把它变成读取已经更新的邻居。

```
X = [[(10 ** lane) * (k + 1) for k in range(4)] for lane in range(4)]
expected = [sum(row[k] for row in X) for k in range(4)]

# Ordinary all-reduce: keep every component in every lane.
ordinary = [row[:] for row in X]
for stride in (2, 1):
    old = [row[:] for row in ordinary]
    ordinary = [
        [old[lane][k] + old[lane ^ stride][k] for k in range(4)]
        for lane in range(4)
    ]
assert all(row == expected for row in ordinary)

# Reduce-scatter: halve the component window at each round.
values = [row[:] for row in X]
owners = [list(range(4)) for _ in range(4)]
for stride in (2, 1):
    old = [row[:] for row in values]
    old_owners = [row[:] for row in owners]
    half = len(old[0]) // 2
    for lane in range(4):
        start = half if lane & stride else 0
        keep = range(start, start + half)
        peer = lane ^ stride
        assert old_owners[lane] == old_owners[peer]
        values[lane] = [old[lane][k] + old[peer][k] for k in keep]
        owners[lane] = old_owners[lane][start:start + half]
    print("stride", stride, "owners", owners, "values", values)

scattered = [0] * 4
for lane in range(4):
    component = owners[lane][0]
    scattered[component] = values[lane][0]
    print("lane", lane, "owns", component, "sum", values[lane][0])
assert scattered == ordinary[0] == expected
print("equal:", scattered)
```

此例用整数，因此断言可以逐项精确相等。真实浮点梯度的相加顺序改变后，一般只能要求误差在约定容差内，而不能要求逐 bit 相同。在这个教学模型里，普通路径每 lane 需要 $4 + 4 = 8$ 次标量交换，scatter 路径需要 $2 + 1 = 3$ 次；这不是 GPU kernel 时间必定缩短 $8/3$ 倍的证明。寄存器移动、地址计算、atomic、occupancy 与访存都会影响最终收益。

3.1.3 算例三：把 ownership 映射到完整 CUDA warp

下面是**算法伪代码，不是可直接编译的 CUDA 源文件**。它专门处理每 lane 四个梯度分量、完整 32-lane warp 的情形。前两轮拆分分量窗口，后三轮只归约一个标量。所有 lane 在每轮执行相同数量的 shuffle，且读取的 peer 寄存器索引保持一致。

```
// PSEUDOCODE: one Gaussian key shared by all 32 lanes.
// v[0..3] are this lane's four finite gradient contributions.
lane = threadIdx.x % 32;  // assume a one-dimensional block
mask = 0xffffffffu;
```

```
// Exchange matching components before selecting a window.
for k in 0..3:
    peer[k] = __shfl_xor_sync(mask, v[k], 16, 32);
base = (lane & 16) ? 2 : 0;
for k in 0..1:
    tmp[k] = v[base + k] + peer[base + k];

// Split each 2-component window into a scalar.
for k in 0..1:
    peer2[k] = __shfl_xor_sync(mask, tmp[k], 8, 32);
index = (lane & 8) ? 1 : 0;
sum = tmp[index] + peer2[index];

// Finish the reduction within each 8-lane ownership group.
for stride in [4, 2, 1]:
    sum += __shfl_xor_sync(mask, sum, stride, 32);

component = lane >> 3;  // lanes 0..7 own component 0, etc.
if (lane & 7) == 0:
    atomicAdd(global_gradient[gaussian_id][component], sum);
```

为什么不能先选窗口再交换？ 因为 XOR 16 的两个 lane 选择互补窗口，而 shuffle 读取的是对方在同一条指令上提供的值。若直接交换 `v[base + k]`，对方提供的可能恰好是另一个分量。上面的代码先交换同编号分量，再在本地选择窗口，避免了这个陷阱。

这个安全教学版本优先表达正确交换关系，不宣称达到最少 shuffle 指令数。更高效的 reduce-scatter 会设计寄存器排列、选择与交换的联合协议；不能把 Python 的随机访问 `old[peer][k]` 机械翻译成 lane 各自选择源寄存器的 shuffle。

完整 mask 是执行契约。 `0xffffffffu` 要求 mask 指定的所有尚未退出线程执行匹配的 collective，而且被读取的源 lane 必须确实参与。不能让某些像素线程在 shuffle 前 `continue`，剩余线程却仍用 full mask。像素无效时可以让线程保留参与资格并提供零梯度，但零值必须在危险计算之前产生。此外，所有参与线程应当归约同一个 Gaussian 的同一组参数；若 Gaussian ID 不同，数学上就需要分组归约而不是直接相加。

对线程数不是 32 倍数的 block，最后一个 warp 可能不完整。例如 4×4 block 只有 16 个线程，不能使用上述 full-warp 算法。保守派发方式是：完整 warp 专用路径仅用于满足其前提的配置，其余配置回退到已验证的通用 reduction。通用路径必须正确处理部分 warp、参与 mask 与分量写回；仅将 full mask 替换为 `__activemask()` 并不能修复读取缺失 XOR partner 的问题。二维 block 还需用线性线程编号计算 lane，而不是直接套用上面的 `threadIdx.x % 32`。

3.1.4 算例四：Amdahl 模型解释完整渲染为什么只快一点

设基线完整渲染时间为 F，其中被优化的 raster 阶段占 R，其他阶段是 $U = F - R$。若 raster 的加速比为 s，并暂时假定其他阶段不变，则

$$F' = U + R/s, \qquad S_{\text{full}} = \frac{F}{F'} = \frac{1}{(1-f) + f/s}, \qquad f = R/F.$$

例如 $F = 20$ ms、$R = 6$ ms、$s = 1.5$，新的 raster 是 4 ms，完整时间是 18 ms，完整加速仅为 1.111 倍。如果忽略其余 14 ms，把 1.5 倍写成完整训练或完整渲染收益，就扩大了结论。即使 raster 变成零耗时，这个例子的上限也只有 $20/14 \simeq 1.429$ 倍。

```
from math import isclose

def full_render_model(full_ms, raster_ms, raster_speedup):
    assert 0.0 <= raster_ms <= full_ms and full_ms > 0.0
```

```
    assert raster_speedup > 0.0
    unchanged_ms = full_ms - raster_ms
    candidate_ms = unchanged_ms + raster_ms / raster_speedup
    return candidate_ms, full_ms / candidate_ms

new_ms, speedup = full_render_model(20.0, 6.0, 1.5)
assert isclose(new_ms, 18.0)
assert isclose(speedup, 20.0 / 18.0)
for raster_ms in (1.0, 6.0, 15.0):
    new_ms, speedup = full_render_model(20.0, raster_ms, 1.5)
    print("raster baseline:", raster_ms,
          "full candidate:", round(new_ms, 3),
          "full speedup:", round(speedup, 3))
```

模型有意省略 kernel overlap、CPU 提交、缓存、功耗与频率变化。CUDA events 测得的阶段时间相加未必等于有重叠流水线的端到端墙钟时间。因此 Amdahl 是一致性检查，不是代替实测的性能预测器。若优化只针对 backward，也应将 R 定义为被优化的 backward 部分，不能把整个 forward 加 backward 的时间都当成可以按 s 缩短的部分。

3.1.5 算例五：对齐 benchmark，并计算 relL2

先固定比较对象。 两次测试应共享相同的 Gaussian 参数、相机、分辨率、排序输入和 loss 上游梯度。还需固定 dtype、SH degree、channel 数、packed 模式、背景、absgrad、裁剪阈值、tile 大小与输出类型，例如 RGB 或 RGB 加 expected depth。输出深度与颜色的量纲不同，宜分别报告误差，避免某个大尺度通道掩盖另一通道。测量之前完成 CUDA 编译与预热；GPU 计时使用同一 stream 上的事件并等待完成，或在端到端墙钟计时边界正确同步。不要拿异步 launch 的 CPU 返回时间与实际执行完毕的 GPU 时间相比。

对参考向量 x 和候选向量 y，这里明确采用

$$\mathrm{relL2}(y,x)=\frac{\sqrt{\sum_i (y_i-x_i)^2}}{\max\left(\sqrt{\sum_i x_i^2},\varepsilon\right)},\qquad \varepsilon=10^{-12}.$$

分母使用参考向量的范数；这是整体张量指标，不是逐元素相对误差的平均值。例如 $x=(3,4)$、$y=(3.03,3.96)$，差的范数是 0.05，参考范数是 5，因此 relL2 是 0.01，即 1%。同时报告最大绝对误差 0.04，能补充整体范数可能稀释局部错误的问题。

```
from math import isclose, isfinite, sqrt

def compare(reference, candidate, eps=1e-12):
    assert len(reference) == len(candidate) and len(reference) > 0
    assert eps > 0.0
    if not all(isfinite(v) for v in reference + candidate):
        raise ValueError("non-finite input")
    error = [b - a for a, b in zip(reference, candidate)]
    error_norm = sqrt(sum(v * v for v in error))
    reference_norm = sqrt(sum(v * v for v in reference))
    return (error_norm / max(reference_norm, eps),
            max(abs(v) for v in error))

rel, maximum = compare([3.0, 4.0], [3.03, 3.96])
assert isclose(rel, 0.01, rel_tol=1e-12)
assert isclose(maximum, 0.04, rel_tol=1e-12)
print("relL2:", rel, "max_abs:", maximum)
zero_rel, zero_abs = compare([0.0, 0.0], [1e-9, 0.0])
print("zero-reference case:", zero_rel, zero_abs)
```

零参考例子的 relL2 为 1000，但绝对误差只有 10^{-9}。因此不能只看一个相对阈值，更不能把非有限值静默替换成零之后再宣布通过。对大型数组应使用数值稳定的范数计算与足够精度的累加；上面的小程序只是手算验证。实践中分别检查 forward、各组参数梯度和 absgrad，额外检查空输入、完全遮挡、阈值边界与部分 warp 配置。浮点归约顺序引起的小误差，与 ownership 错位造成的大误差，应通过这些局部比较来区分。

时间统计也要对齐：同一工作负载重复多次，报告预热后的中位数与离散程度，并区分 raster-only 与完整路径。每个相机先计算自己的比值，再明确选择总耗时比、算术平均或几何平均；这些汇总不是同一个统计量。

3.1.6 算例六：安全 predication 不是最后乘一个零

危险来自未选中路径的中间量。 考虑单精度浮点数中的无效候选 $\sigma = -100$。若先无条件计算 $\exp(-\sigma) = \exp(100)$，结果会溢出为无穷大。即使最后令 alpha 为零，再做 $0 \times \exp(100)$ 仍得到 NaN，而不是零。同理，即使每个输入都有限，两个约 10^{30} 的数相乘也会在 float 中溢出。因此 “输入 finite” 并不推出 “梯度乘积 finite”。

下面是**合法 CUDA C++ 设备函数片段**，需要 CUDA 编译器；不是完整的 kernel 或独立可执行程序。它演示一个保守策略：先验证指数参数，再计算 alpha；在普通 alpha 链式梯度路径中，上限夹紧的样本不更新 opacity 与几何梯度，但仍保留颜色梯度。实际系统须沿用自身对夹紧与阈值边界的导数约定。

```
#include <cuda_runtime.h>
#include <math.h>

__device__ bool safe_alpha(float sigma, float opacity,
                           bool inside, float* alpha,
                           float* exponential, bool* grad_alpha) {
    *alpha = 0.0f;
    *exponential = 0.0f;
    *grad_alpha = false;
    if (!inside || !isfinite(sigma) || sigma < 0.0f ||
        !isfinite(opacity) || opacity < 0.0f || opacity > 1.0f) {
        return false;
    }
    float e = expf(-sigma);  // non-positive exponent: no overflow
    float raw = opacity * e;
    if (raw < 1.0f / 255.0f) {
        return false;
    }
    *alpha = fminf(raw, 0.999f);
    *exponential = e;
    *grad_alpha = raw < 0.999f;
    return true;
}

__device__ bool finite_product(float x, float y, float* result) {
    *result = 0.0f;
    if (!isfinite(x) || !isfinite(y)) {
        return false;
    }
    double p = static_cast<double>(x) * static_cast<double>(y);
    if (!isfinite(p) || fabs(p) > 3.4028234663852886e38) {
        return false;
    }
    *result = static_cast<float>(p);
    return true;
}
```

其中 `safe_alpha` 的 bool 返回值表示是否保留该样本的颜色贡献，`grad_alpha` 单独表示是否进入未夹紧的 alpha 导数路径。不能把这两种有效性合并，否则夹紧样本的颜色梯度会被错误删除。`finite_product` 用 double 检查两个有限 float 的乘积是否可表示为 float；它仅用于说明保护方式，不建议无测量地放进所有热路径。返回 false 时调用方应报告异常、触发诊断或走明确的回退路径，不能把溢出的合法梯度静默视为零。连续多个乘法与累加也需要自己的范围分析：保护单次乘法并不保护后续求和或 atomic 累积。

与 warp collective 共存。 在实际后向循环里，可以让无效 lane 的局部梯度寄存器初始化为零，只在有效分支内计算危险的几何、opacity 和 absgrad 表达式，随后所有需要参与的 lane 在分支外执行归约。不要让设备函数的局部返回变成调用 kernel 的提前退出；也不要为消除分支而重新把危险表达式搬到有效性判断之前。数值安全来自控制数据依赖与计算范围，不来自代码表面上没有 `if`。

最后的自检顺序。 先用第一个例子验证 conic、alpha 与前向合成；再用第二个例子固定分量 ownership；随后检查 full mask、部分 warp 与无效 lane 的执行契约；最后对齐误差和计时口径，用 Amdahl 模型检查完整路径的收益是否合理。这条顺序把数学错误、线程协作错误、数值错误和统计错误拆开，避免用一个看起来漂亮的加速比掩盖其中任何一种问题。

Chapter 4

gsplat 移植、评估和最终结论

4.1 从文章到 gsplat：保留的是哪些实现

优化移植不是把一张性能表搬到另一套代码里，而是重新确认数据布局、执行所有权和数值契约。本章以实际交付的 gsplat 提交 88513ea 为对象：最终仅保留后向 rasterizer 的窄通道 reduce-scatter、完全遮挡后缀裁剪和安全 alpha predication。前向 rasterizer 与 SH 源码未保留实验性改动。**最终计时只采用重建二进制的新证据**：benchmarks/blog_eval/results/REBUILT_FINAL_SUMMARY.md；benchmarks/blog_eval/REPORT.md 的后半部分保留了实验历史，不能把其中旧表格混入最终结果。尤其是存在 partial-warp 缺陷的 b96efa8 已退役。

4.1.1 先找到源码中的执行边界

以下位置均相对于仓库根目录。核心 kernel 是 submodules/gsplat/gsplat/cuda/csrc/rasterize_to_pixels_bwd.cu 中的 rasterize_to_pixels_bwd_kernel；同文件的 call_kernel_with_dim 按通道数实例化、分配零初始化梯度，并在当前 CUDA stream 上启动 kernel。每个 block 对应一个相机的一个 tile，每个线程对应一个像素；线程同时协作从 flatten_ids 加载一批 Gaussian 到 shared memory，然后逆深度顺序处理。图像边缘之外的线程仍要参与加载、同步和归约，不能直接逐线程退出。

后向恢复的是前向已经决定的贡献序列。对当前 Gaussian，若前向更新为 $T_{\text{after}} = T_{\text{before}}(1-\alpha)$，则反向用

$$T_{\text{before}} = \frac{T_{\text{after}}}{1-\alpha}$$

恢复透射率。实现还维护后方颜色贡献的 buffer[COLOR_DIM]。一个 warp 的多个像素会更新同一个 Gaussian，因此先在 warp 内累加，再用原子加写回全局梯度。优化改变这些局部和的计算与发出方式，不改变前向 alpha 上限 0.999、低贡献阈值 $1/255$、像素中心或终止语义。

交叉阅读时还应查看 submodules/gsplat/gsplat/cuda/csrc/isect_tiles.cu：此 fork 已有 opacity-aware compact-box 与 sweep 枚举。compact_box_impl="sweep" 是算法选择，compact_box=True 才是启用开关；既有能力不能再次记成此次移植的新收益。

4.1.2 Reduce-scatter：把结果所有权分散到 lane

设 $x_{\ell,k}$ 是 lane ℓ 对分量 k 的贡献，目标为

$$s_k = \sum_{\ell=0}^{31} x_{\ell,k}.$$

原路径对每个分量执行 warpSum，最后 lane 0 发出所有 atomic add。新的 helper 位于 submodules/gsplat/gsplat/cuda/include/warp_reduce_scatter.cuh，符号为 WarpReduceScatter<DIM, STRIDE>::run。它不是减少数学上的求和项，而是避免每个 lane 都保存所有分量的最终副本。

实际归约向量不是固定九维。 在本次实现中，$C = \texttt{COLOR_DIM}$，$D = C + 8$：依次放入 C 个颜色梯度、三个 conic 梯度、两个屏幕均值梯度、一个 opacity 梯度和两个可选 absgrad 分量。所以 RGB 使用 $D = 11$，RGB 加深度使用 $D = 12$。即使关闭 absgrad，这两个槽位仍初始化为零，owner 最终只在目标指针存在时写出。不能把文章中九维示意直接当作此 kernel 的寄存器布局。

每一层同时划分 lane 与分量。 递归从 STRIDE=16 开始，令 $H = \lceil D/2 \rceil$。下半组 lane 保留低分量窗口、发送高窗口；上半组反过来，通过 XOR shuffle 交换后相加。上半组的 component 加 H，然后递归处理大小为 H 的窗口与更小 stride。RGB 的活跃窗口依次为

$$11 \longrightarrow 6 \longrightarrow 3 \longrightarrow 2 \longrightarrow 1.$$

到一维时仍需完成剩余 lane 组的求和，并选出唯一 owner。奇数窗口补出来的槽位必须持续标为无效；源码中的 previous_component 与 owner=false 防止 padding 槽位别名到真实分量。最终只有合法 owner 按 component 选择目标地址并 atomic add。

这里的证明义务有三个：每个分量恰有一个写出者；其值包含完整 warp 的全部贡献；不存在 padding 写入或错误地址。原子加次数并未神奇消失，只是从 lane 0 串行发出多个分量，变为不同 owner 分散发出。浮点求和顺序也随之变化，故后向不应承诺逐位相同。静态指令和寄存器信息能解释优化意图，却不能替代运行时间与梯度对齐测试。

4.1.3 4×4 tile 暴露的 partial-warp 错误

最初的测试主要使用 8 或 16 边长 tile。随后直接测试 4×4 tile：block 只有 16 个线程，旧候选把基线约 17.697 的非零 opacity 梯度错误变成零。原因不只是 shuffle mask 写错，而是完整 32-lane 算法把某些分量的所有权交给了不存在的上半组 lane。把 full mask 改成 active mask，并不能凭空创建这些 owner 或重新分配它们。

交付版本使用下面的实际分派原则；它位于 kernel 的窄通道归约分支：

```
if constexpr (COLOR_DIM <= 4) {
    if (block_size % 32 != 0) {
        standard_reduce();
        continue;
    }
    // Full-warp reduce-scatter and owner atomics.
} else {
    standard_reduce();
}
```

standard_reduce 是原有逐分量归约和 lane-0 原子写回。条件检查整个 block 的线程数：6×6 tile 有 36 线程，虽然包含一个完整 warp，也将整个 block 的归约切回旧路径，避免最后四个线程套用完整 warp 的所有权模型。宽通道一直走旧归约。这个保守 fallback 用少量特殊尺寸的性能换取清楚的有效域，不是声称 helper 已经支持任意 active mask。

4.1.4 后缀裁剪：从像素停止位置推导安全加载终点

last_ids[p] 保存像素最后贡献者的绝对 intersection 位置，不是 Gaussian ID，也不是 tile 内相对下标。原 kernel 虽然在像素循环中跳过停止位置之后的元素，但仍可能把整段被遮挡的尾部搬进 shared memory。保留的优化先求 warp 最大停止位置，再由各 warp 的 lane 0 对 shared-memory 槽执行 atomicMax，用 block 同步保证所有线程读到同一终点。

若原 tile 范围为 $[a, b)$，代码先把最大值槽初始化为 a，因此实际使用

$$m = \max\big(a,\ \max_{p\ \text{inside}} \texttt{last_ids}[p]\big), \qquad b' = \min\big(b, \text{int64}(m) + 1\big).$$

这个保守初始化并非保证所有空贡献 tile 都零加载；它保证不把范围裁到起点之前。只有 $[b', b)$ 这个所有像素都不会用到的后缀被省略，内部不贡献者仍在列表中。原先的 warp 停止位置跳过仍保留，两者分别减少 staging 和像素内遍历。

两个边界值得逐行检查。第一，$a = b$ 的空范围必须在 collective 和独占终点运算之前 tile-uniform return；不能让部分线程先走而其他线程等待 block barrier。第二，源码先把最大位置转换到 `int64_t` 再加一、取最小值，最后才转回 `int32_t`，避免接近 `INT_MAX` 时先发生有符号溢出。裁剪不移动 `flatten_ids`，不压缩 ID，也不更改原来的绝对停止位置，所以其他 channel chunk 重用同一 intersection list 时仍然有效。

这不是文章的完整 contributor compaction。文章的 24-bit ID 加 8-bit mask 方案不能直接用于 $20{,}789{,}814 > 2^{24}$ 的场景；最终记录中实际访问的 ID 达到 $20{,}789{,}812$。安全的完整移植需要独立 mask、更宽存储或显式重映射，而不是截掉 Gaussian ID 的高位。

4.1.5 安全 predication：无效 lane 的零必须是真的零

窄通道路径让 miss 使用 $\alpha = 0$，透射率恢复成为恒等操作：$T \leftarrow T/(1-0) = T$。这能减少逐 lane 的控制流分歧，但**不等于把整个梯度公式无条件执行**。即使输入均有限，conic 与位移相乘也可能溢出：

$$\underbrace{0}_{\text{屏蔽系数}} \times \underbrace{(a\,d_x + b\,d_y)}_{\text{可能成为 Inf}} = \text{NaN},$$

之后的 warp 求和会把 NaN 传播给有效像素。类似地，拒绝 lane 的非有限上游颜色梯度不能通过乘零来清除。

实际代码把局部梯度初始化为零，并分开控制三件事：

- 颜色梯度使用 `valid ? fac * v_render_c[k] : 0.f`，而不是无条件乘上零系数；颜色 buffer 只在 `valid` 时更新。
- 几何、opacity 与 absgrad 只在 `valid && opac * vis <= 0.999f` 时计算；clamp 之外保留初始化零。被 clamp 的有效 Gaussian 仍可能贡献颜色，不能把这两种有效性混为一谈。
- `warp.any(valid)` 为假时，整个 warp 继续跳过当前 Gaussian。此 fork 没有文章中保证存在贡献者的精确 subtile mask，不能删除这个 skip。

某些中间标量仍可在无效 lane 上计算，但不会进入梯度写回或持久 buffer。因此这里的 “branchless” 应理解为对特定 recurrence 的 predication，不是禁止一切分支，更不是放弃非有限数隔离；宽通道仍采用原条件执行路径。

4.2 拒绝实验也是结果，而不是漏掉的步骤

4.2.1 三个已测试、未保留的候选

下列时间均来自报告中的历史消融序列，仅说明当时为何拒绝相应改动，不是重建最终二进制的增益分解，也不能与后文最终基线交叉作比。

标量颜色累积。 把多通道后方颜色 buffer 投影到固定上游颜色梯度，可把向量递推改成标量递推，同时把背景点积移出循环。代数等价不保证机器上更快：112 个小例通过，但该序列 raster 中位数由 13.945 ms 变为 14.496 ms，没有实测优势，因此撤销。该候选单次 raw-quaternion 误差也未通过严格门限，不能只引用小例通过。证据为 `benchmarks/blog\_eval/results/regression-scatter-trim-scalar.json` 与 `benchmarks/blog\_eval/results/scatter-trim-scalar-normalized-alignment.json`，计时文件为同目录的 `scatter-trim-scalar-raster-job*.json`。

SH selective zeroing。 submodules/gsplat/gsplat/cuda/csrc/compute_sh_bwd.cu 调用 sh_coeffs_to_color_fast_vjp，按 $N \times 3$ 线程分配颜色通道并直接写系数梯度；不是文章中单线程同时持有 48 个系数和 48 个梯度的起点。实验把梯度 bulk zeroing 合并到 kernel，保证 masked 行与未使用的高阶系数仍写零。288 个 SH 案例通过，但配对完整渲染的中位数为 $64.056 \rightarrow 64.973$ ms，没有可靠收益，故撤销。索引、除法和谓词开销可能抵消内存写节省，但这是解释假设，不是 profiler 已证实的瓶颈。直接把 zeros_like 改成 empty_like 则是不正确的捷径。参见 benchmarks/blog_eval/results/sh-regression-selective-zero.json 以及 benchmarks/blog_eval/REPORT.md 的 Experiment 4。

前向 AABB warp culling。 原型为 tile16、窄通道使用 8×4 warp 像素区域，生成保守 opacity-aware 椭圆 AABB mask，再由 ballot/ffs 遍历候选。它不是文章的精确 conic row-span mask，也没有移植 48-byte packed splat 布局。FP32 版本三组配对 raster 中位数 $16.525 \rightarrow 16.195$ ms，约 $1.020\times$；收益小且有噪声，不足以承担专用循环、病态 conic fallback 和边界舍入复杂度。审查还发现早期 FP64 bounds 版本对溢出浮点的基线兼容风险。原型已撤销，证据见 benchmarks/blog_eval/results/regression-floatcull.json 及报告 Experiment 6。拒绝这个近似原型，不能推导出文章的精确 mask 无效。

4.2.2 没有实现，就没有可引用的收益

两 Gaussian folding 尚未实现；当前归约仍是单 Gaussian。两阶段排序也仅做源码评估：窄 tile key 有潜力，但须保留等深度稳定顺序、处理多相机与超过 16-bit 的 tile 网格，并提供 fallback。完整 contributor compaction、精确 conic subtile mask、packed splat 属性结构均未交付。multirow 与 pipe balancing 不能直接套用 A100 的 occupancy 或寄存器限制到 Ada。现有 __expf 配合 --use_fast_math 已使用快速指数路径，改个 exp2 名字不能算新的测得加速；系数重标定仍需阈值边界测试。BF16 SH 未实现、未做质量验证，会改变数值与优化器契约。本场景只有 degree-1 SH，系数载荷本来就只有 degree-3 的四分之一。

4.3 机器评估：先证明比较的是同一个问题

4.3.1 输入、二进制与计时协议

最终环境为 RTX 4090（SM89）、PyTorch 2.10.0+cu128、CUDA 12.8；场景含 $20{,}789{,}814$ 个 Gaussian，分辨率为 1280×720，degree-1 SH。虽然 checkpoint 配置来自 PGSR，本实验测的是 gsplat 标准 3DGS API，不包括 PGSR 专用平面深度或法向 kernel。基线 gsplat 为 ffe2db3，交付源码为 88513ea。候选文件 final-partial-warp-safe.so 的 SHA-256 为

```
6160354698af48c34a5e6a090c5e241c564959fb6e0d54d2c55bb3d9c8373ddb
```

benchmarks/blog_eval/results/build_manifest.json 与 benchmarks/blog_eval/results/final_verification.json 保存身份和验证关系。运行时加载文件的 hash 比单独记录 checkout HEAD 更直接；HEAD 本身不构成独立的可复现构建证明。

参数域错误会使整张性能表失效。 benchmarks/blog_eval/gsplat_blog_benchmark.py 的 full_render 在调用 gsplat 前执行

$$s = \exp(s_{\log}), \qquad o = \mathrm{sigmoid}(o_{\log}), \qquad \widehat{q} = \frac{q}{\|q\|}.$$

早期直接传 log-scale 的探索数据已被判无效，不能引用。相机使用 camera-to-world 的逆变换；四元数输入额外归一化以匹配产品 wrapper，即使 gsplat 内部也做归一化，原始参数的计算图仍应一致。记录的 grad.quats 是对原始四元数张量的梯度，不是把归一化结果当独立叶子后的梯度。

完整渲染和 raster-only 都包含前向加后向。 `Full` 包含参数变换、投影、SH、binning/sort、raster 和反向传播，不包括 optimizer、训练损失、数据读取和整个训练循环。`Raster` 用一次未计时、无梯度的完整渲染取得相同投影与 intersection 元数据，把屏幕均值、conic、颜色和 opacity detach 成叶子，再测 raster API 的前后向。RGB+ED 还保留深度除以 alpha 的 expected-depth 后处理及其反向；它不是单个 CUDA backward kernel 的裸计时。

所有最终任务使用全量点、同一个 compact-box sweep、相同相机与随机种子，对图像和 alpha 施加固定随机上游梯度。必须显式核对 `compact_box` 与 `compact_box_impl`：当前 harness 的 reference signature 未涵盖全部这些开关，不能只靠 signature 相等排除配置错误。fixed-16 检查曾在采集候选前发现 baseline-only 配置关闭 compact-box，已统一为 true/sweep 并重新生成两份基线，未保留不匹配对照。

计时使用 CUDA events，warmup 后同步；逐轮清除梯度与旧输出图，避免图累积影响显存和耗时。最终运行串行进行，没有同时编译、安装或本任务的其他 GPU 工作负载，也未操纵 GPU 时钟。`--timing-only` 的最终 finite 检查只说明没有非有限输出，不是梯度对齐检查。所有 36 个最终计时任务均通过该 finite 检查。

4.3.2 重建后的最终时间：RGB 三个相机

每个相机、每个模式独立执行两对 baseline/candidate，顺序为 baseline 1、candidate 1、baseline 2、candidate 2；每个任务 40 次 warmup、100 次计时。表中时间是任务中位数的中位数，speedup 为两列时间之比，不是把不同视角或历史运行混在一起平均。

相机	模式	基线/ms	候选/ms	加速比	两对各自加速比
0	Raster	23.318528	17.443752	1.337	1.282 / 1.392
0	Full	69.148039	62.850912	1.100	1.114 / 1.087
2552	Raster	3.269248	1.983760	1.648	1.649 / 1.647
2552	Full	40.359648	39.728473	1.016	1.131 / 0.917
5103	Raster	2.625744	1.926184	1.363	1.325 / 1.402
5103	Full	40.084271	38.574593	1.039	1.011 / 1.069

Raster 的六对均同方向改善。相机 2552 的 Full 一对加速、一对变慢，候选任务中位数为 36.740273 与 42.716673 ms，跨度达其中位数的 15.04%，因此聚合的 1.6% 收益**不能认为结论确定**。另两视角 Full 的配对方向一致，但仅两个任务不足以给出精确不确定度估计。这也说明为何不能把 raster 的局部收益叫作训练加速。原始文件为 `benchmarks/blog\_eval/results/` 下 `rebuilt-paired-<baseline|candidate>-cam<0|2552|5103>-<raster|full>-job<1|2>.json`，共 24 份；每份任务中位数均可在最终 summary 中核对。

4.3.3 产品配置：RGB+ED、absgrad、相机 0

这组显式启用 `--render-mode RGB+ED --absgrad`，每模式三对独立任务，每任务 30 次 warmup、80 次计时，仍使用全部点和 compact-box sweep。

模式	基线/ms	候选/ms	加速比	三对各自加速比
Raster	23.219712	17.717712	1.311	1.310 / 1.321 / 1.350
Full	72.297985	66.900990	1.081	1.063 / 1.103 / 1.067

六对均改善，但 Full 基线和候选任务范围分别约为其中位数的 3.48% 与 5.16%，不能把最后几位小数当稳定物理常数。原始 12 份文件为 `benchmarks/blog\_eval/results/` 下 `rebuilt-production-<baseline|candidate>-<raster|full>-job<1|2|3>.json`。这里的 raster 比值是 $1.311\times$，不是旧 `PRODUCTION_SUMMAR` 的 $1.346\times$；两代二进制证据不能互换。

4.4 正确性必须分层作答

4.4.1 单次梯度：屏幕空间通过，原始四元数仍失败

定义梯度相对误差为

$$E(g) = \frac{\|g_{\text{candidate}} - g_{\text{baseline}}\|_2}{\max(\|g_{\text{baseline}}\|_2, \varepsilon)}.$$

最终检查保持梯度门限 10^{-4}、前向最大绝对误差门限 2×10^{-5}，不因失败而调宽。每个设置各执行一次完整对照检查，camera 0，全点，3 次 warmup、3 次计时；内部 repeat 用于提供噪声诊断，不是不断重试直到通过。

设置	检查域	最大梯度相对 L2	判定
RGB	Raster	2.615650×10^{-6}	通过
RGB	Full	1.628947×10^{-4}	raw quaternion 失败
RGB+ED、absgrad	Raster	1.470557×10^{-6}	通过
RGB+ED、absgrad	Full	1.633092×10^{-4}	raw quaternion 失败

四项图像与 alpha 都与基线逐位相同，比较张量与计时末轮输出、梯度均有限；Full 中其余参数梯度通过。Raster 是对屏幕均值、conic、颜色、opacity，以及开启时的 absgrad 的直接检查，不等同于全部原始参数的检查。证据分别是 benchmarks/blog_eval/results/ 下的 rebuilt-final-RGB-full-alignment.json、rebuilt-final-RGB-raster-alignment.json、rebuilt-final-RGBED-full-alignment.json 和 rebuilt-final-RGBED-raster-alignment.json。

同一候选自身重复的 quaternion 相对 L2 为 2.221081×10^{-4}（RGB）与 1.395373×10^{-4}（RGB+ED）。这与原子累加顺序噪声经病态投影放大的解释一致，但不是数学证明，更不是豁免规则。**噪声背景可以解释为什么继续调查，不能把 JSON 中的失败改称通过。**

4.4.2 预先固定 16 次均值：回答的是另一个问题

补充协议 benchmarks/blog_eval/gsplat_gradient_ensemble.py 预先固定每二进制、每模式 16 次渲染与反向，参数、相机、上游梯度均保持相同，在 CPU 用 float64 按顺序累积均值。不是每次运行一个不同训练步骤，也不是看到失败后增加采样次数。门限保持不变，全部 16 次都计入，未丢弃样本。

均值梯度相对 L2	RGB	RGB+ED、absgrad
世界坐标均值	6.24×10^{-7}	7.36×10^{-7}
原始四元数	4.57×10^{-5}	3.50×10^{-5}
log-scale	1.48×10^{-6}	2.00×10^{-6}
opacity logit	1.77×10^{-7}	9.55×10^{-8}
SH 系数	2.53×10^{-7}	3.13×10^{-7}
绝对屏幕均值梯度	未启用	1.55×10^{-7}

两种模式的均值图像与 alpha 精确一致，均值梯度全部通过。原始记录为 benchmarks/blog_eval/results/ensemble-final-RGB.json 与 benchmarks/blog_eval/results/ensemble-final-RGBED.json，对应 baseline 文件也在同目录。这为平均梯度没有明显系统偏移提供补充证据，但不是置信区间，不保证每一步满足门限，不推翻单次失败，更不证明优化轨迹或训练收敛相同。

4.4.3 136 个差分案例与两个 sanitizer

benchmarks/blog_eval/gsplat_raster_regression.py 的最终矩阵在默认与非默认 stream 分别通过 136/136；新增 tile4 的 1/3/4/8 通道、absgrad 开关与 tile6 案例，正是为了封住 partial-warp 的覆盖缺口。其他覆盖包括 tile8/16、背景梯度、非对齐图像边缘、空输入与空 intersection、多 batch

tile 列表、packed 相机布局、channel-list 重用、稀疏 tile、重遮挡、opacity cutoff/clamp、有限输入造成的 conic 溢出与被拒绝 lane 的非有限上游梯度。非默认 stream 测试还包含同 stream 的输入生产和梯度消费。

重建最终二进制的整个 136-case 矩阵也通过 CUDA compute-sanitizer memcheck 与 synccheck，均为 0 errors。差分结果见 `benchmarks/blog\_eval/results/partial-warp-final-default.json` 与 `benchmarks/blog\_eval/results/partial-warp-final-nondefault.json`；sanitizer 的 JSON 与日志为同目录 `partial-warp-final-memcheck.*`、`partial-warp-final-synccheck.*`。它们分别约束内存访问与同步错误，不是穷尽数值证明，也未运行 racecheck。已知的基线 masked-tile alpha 初始化与完整 SH 路径的默认 stream 问题不在此处修复；直接 raster 的非默认 stream 通过，不能推广为整个渲染器的 stream 安全证明。

最后能说什么？ 可以说：在这一个 degree-1、20.8M 点场景的 RTX 4090 实测中，三个 RGB 视角的 raster 配对收益稳定出现，产品 RGB+ED/absgrad 的相机 0 也有复现；交付代码具有 partial-warp fallback，屏幕空间梯度与固定次数均值检查通过。同时必须说：两项 Full 单次 raw-quaternion 门限仍失败，未做训练收敛实验，只测了 5104 个相机中的三个，产品配置只测一个。**这既不是 A100 文章 headline 的复现，也不是普遍的全训练加速或已验证收敛的生产发布。**可靠的优化结论应把实现有效域、实际收益和仍未满足的门槛一起交付。

延伸阅读与复现

- 原文章：https://grgkopanas.github.io/3dgs_cuda_blog/。
- 工程结果：benchmarks/blog_eval/results/REBUILT_FINAL_SUMMARY.md、benchmarks/blog_eval/results/f
- 回归脚本：benchmarks/blog_eval/gsplat_raster_regression.py；固定重复平均梯度脚本：benchmarks/blog_eval/gsplat_gradient_ensemble.py。
- 给定环境：<your-existing-python-environment>/bin/python；运行时通过 PYTHONPATH=.（在 gsplat 仓库根目录运行）和 --extension 明确隔离 baseline/candidate，不能在同一 Python 进程加载两个 pybind CUDA extension。